

Diabetes Prediction - Ensemble Models vs Results from the Paper's Logistic Regression Baseline

```
import warnings

from sklearn.neighbors import KNeighborsClassifier

warnings.filterwarnings("ignore")
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split, GridSearchCV,
StratifiedKFold
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (
    accuracy_score, confusion_matrix, roc_curve, auc,
    precision_score, recall_score, f1_score,
    precision_recall_curve, average_precision_score,
    classification_report
)
from sklearn.ensemble import StackingClassifier

%matplotlib inline

from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import (GradientBoostingClassifier,
ExtraTreesClassifier)
from sklearn.naive_bayes import GaussianNB

# Load the dataset
dbdf = pd.read_csv('Diabetic.csv')
dbdf_copy = dbdf.copy()
# Display the first few rows of the dataset
dbdf.head()
```

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	
BMI \						
0	6	148	72	35	0	33.6
1	1	85	66	29	0	26.6
2	8	183	64	0	0	23.3
3	1	89	66	23	94	28.1
4	0	137	40	35	168	43.1

	DiabetesPedigreeFunction	Age	Outcome
0	0.627	50	1
1	0.351	31	0
2	0.672	32	1
3	0.167	21	0
4	2.288	33	1

EDA

Critical risk: The EDA fails to detect biologically implausible zeros, which act as de facto missing values.

Impact: Models trained on raw zeros will learn incorrect patterns (e.g., "low glucose = no diabetes"). Action: Replace zeros with medians or use domain-specific imputation (e.g., clinical norms). Preprocessing decision: Must explicitly handle zeros in Glucose, Insulin, BloodPressure, SkinThickness, BMI.

```
# Check for missing values
print(dbdf.isnull().sum())

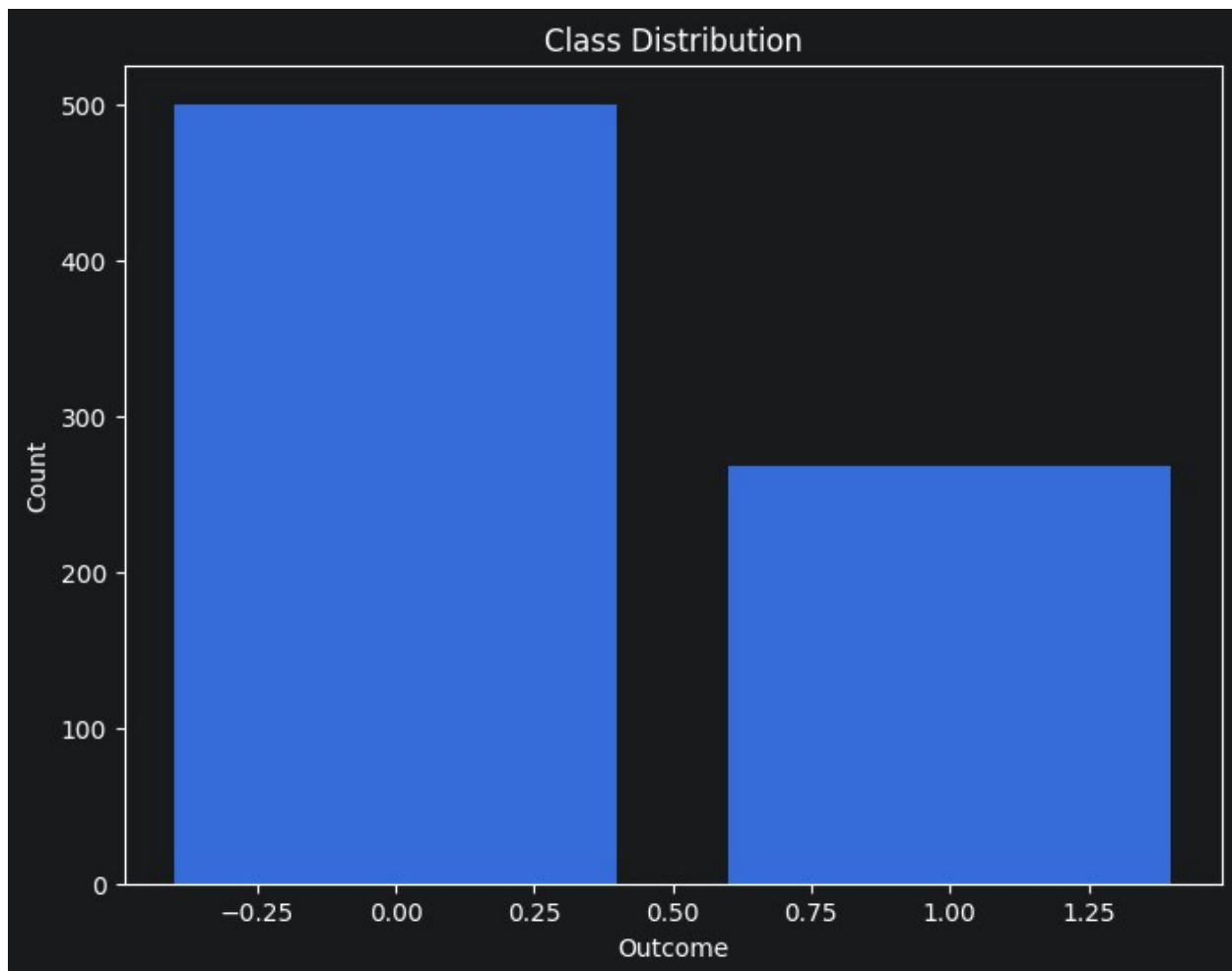
#Check if need to be done only for traning data- Fill missing values
with the median of each column
#dbdf.fillna(dbdf.median(), inplace=True)

# Check for class imbalance
print(dbdf['Outcome'].value_counts())

Pregnancies      0
Glucose           0
BloodPressure     0
SkinThickness     0
Insulin           0
BMI               0
DiabetesPedigreeFunction  0
Age              0
Outcome          0
dtype: int64
Outcome
0      500
1      268
Name: count, dtype: int64

# Visualize class distribution
plt.figure(figsize=(8, 6))
plt.bar(dbdf['Outcome'].value_counts().index,
dbdf['Outcome'].value_counts().values)
plt.xlabel('Outcome')
plt.ylabel('Count')
```

```
plt.title('Class Distribution')
plt.show()
```



Observations:

Clearly there is a class imbalance in the dataset, with more negative cases (Outcome=0) than positive cases (Outcome=1). This imbalance can affect model performance, especially for metrics like accuracy, which may be misleading. We will need to consider this when evaluating our models and may want to use metrics like precision, recall, and F1-score in addition to accuracy.

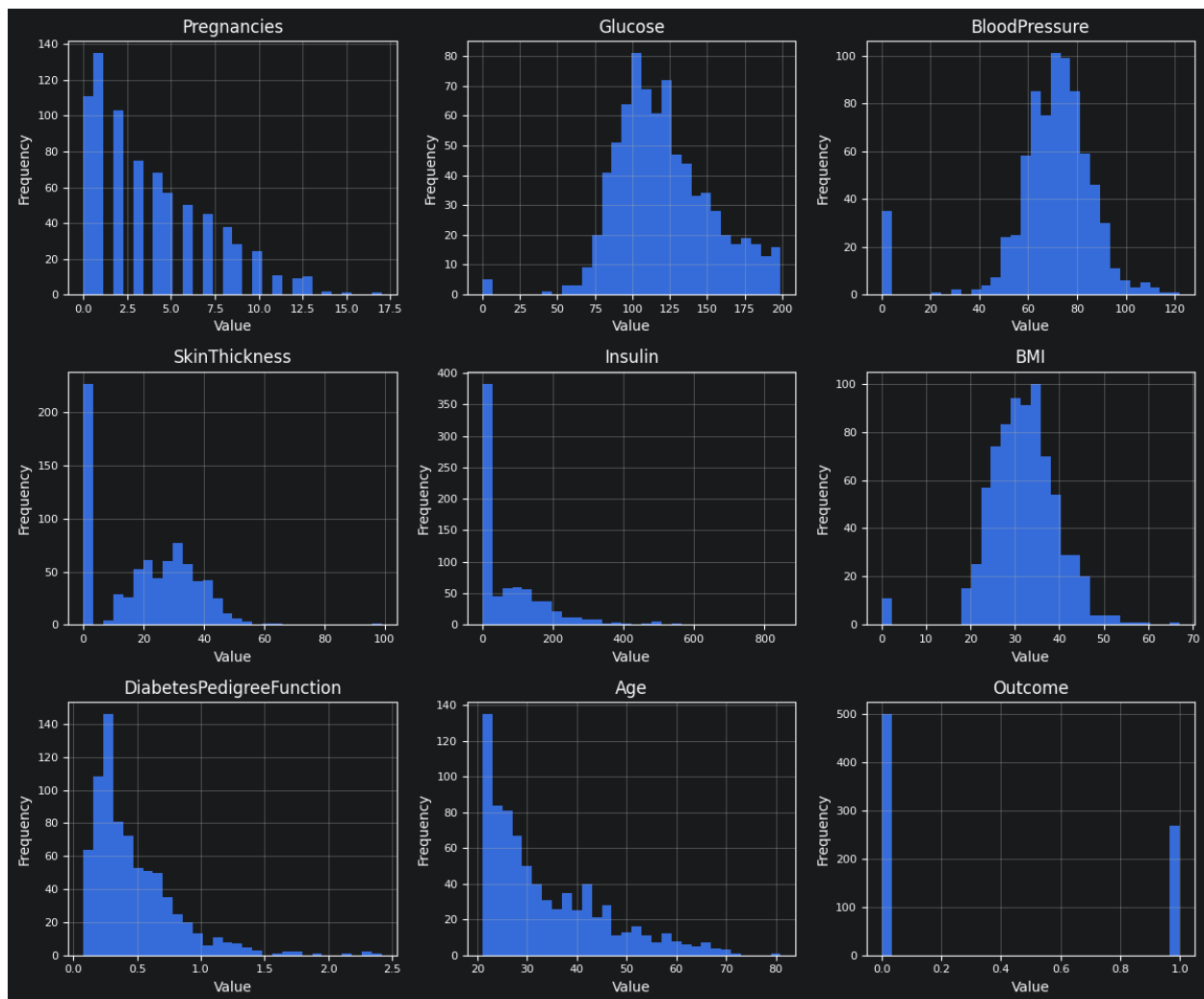
```
axes = dbdf.hist(
    sharex=False,
    sharey=False,
    figsize=(12,10),  # increase figure size
    bins=30           # more intervals
)
```

```

for ax in axes.flatten():
    ax.set_title(ax.get_title(), fontsize=12)
    labels (column names)
    ax.set_xlabel("Value", fontsize=10)
    ax.set_ylabel("Frequency", fontsize=10)
    ax.tick_params(axis='both', labelsize=8)

plt.tight_layout()
plt.show()

```



Observations:

- The features have varying distributions, with some showing skewness (e.g., Glucose, Insulin) and others being more normally distributed (e.g., Age).
- The presence of outliers is evident in some features (insulin, skinThickness), which may need to be addressed during preprocessing.

- The class imbalance is visually confirmed, with a higher frequency of negative cases (Outcome=0) compared to positive cases (Outcome=1).

```
## TODO Quantitive measures (skewness/ kurtosis)
```

```
print(dbdf.skew()) # Check skewness
```

```
print(dbdf.describe().loc[['min', 'max']]) # Check ranges
```

```
Pregnancies      0.901674
Glucose           0.173754
BloodPressure    -1.843608
SkinThickness     0.109372
Insulin           2.272251
BMI              -0.428982
DiabetesPedigreeFunction  1.919911
Age              1.129597
Outcome           0.635017
dtype: float64
```

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI
\ min	0.0	0.0	0.0	0.0	0.0	0.0
max	17.0	199.0	122.0	99.0	846.0	67.1

	DiabetesPedigreeFunction	Age	Outcome
min	0.078	21.0	0.0
max	2.420	81.0	1.0

Observations:

- Nothing is symmetrical (= 0)
- BMI and BloodPressure are negative skew. Negative value (< 0): The distribution is left-skewed. The tail on the left side is longer or fatter. Most data points are clustered on the higher end, with a few exceptionally low values pulling the mean down.
- Highly Right Skewed (if values > 1) Insulin (2.27) and DiabetesPedigreeFunction (1.92): These are heavily right-skewed. The
- Glucose (.17) and skinThickness are close to 0 (Fairly symmetrical values -.5 to .5)

Observations :

Apply RobustScaler scaling for skewed features

Why RobustScaler Over StandardScaler?

Outliers: Features like Insulin have extreme values (skewed distribution). RobustScaler uses median/IQR, which is less sensitive to outliers than StandardScaler (mean/std).

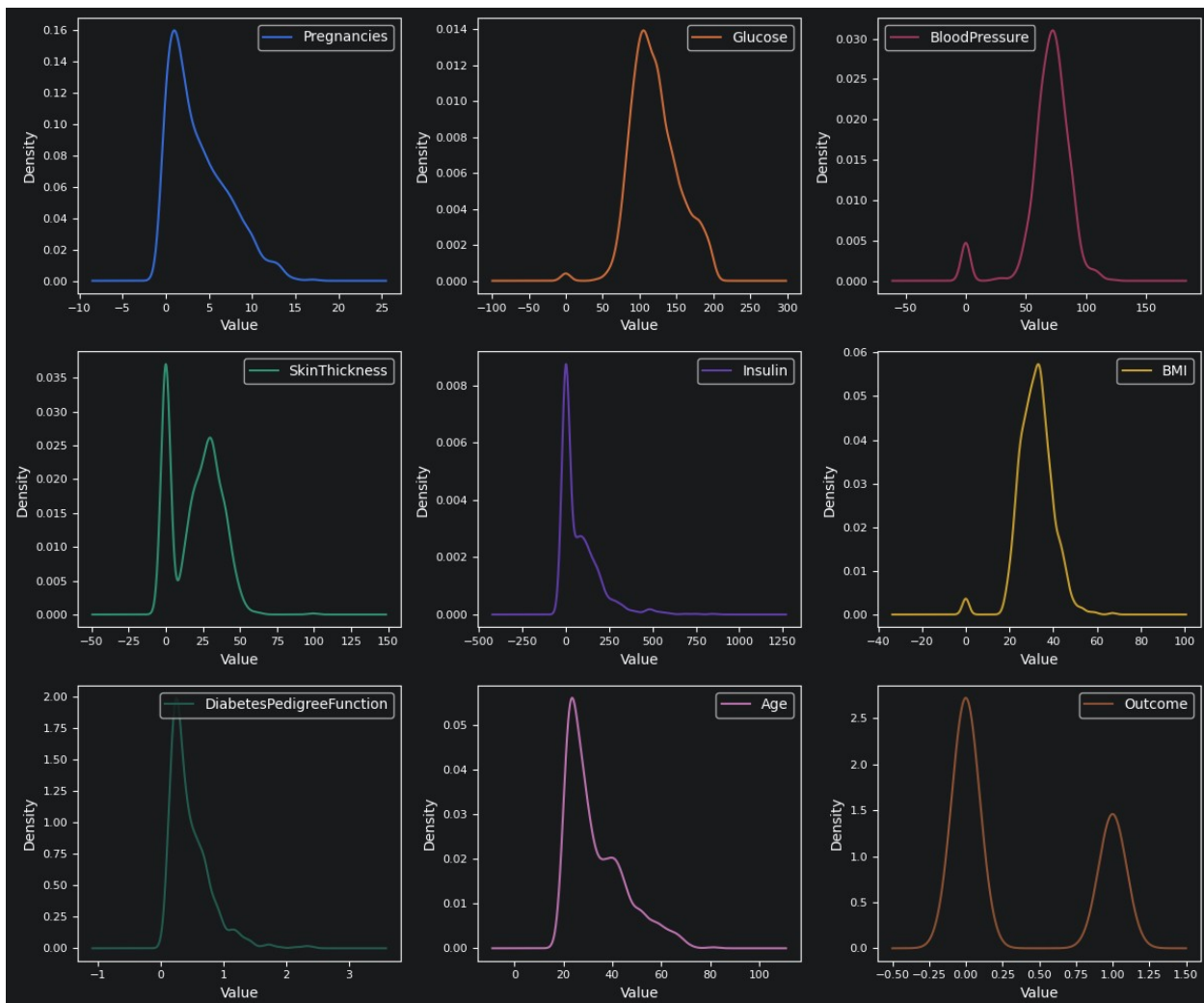
```

axes = dbdf.plot(
    kind='density',
    subplots=True,
    layout=(3,3),
    figsize=(12,10),
    sharex=False,
    sharey=False,
    bw_method=.2
)

for ax in axes.flatten():
    ax.set_xlabel("Value")
    ax.set_ylabel("Density")
    ax.tick_params(axis='both', labels=8)
    ax.set_title(ax.get_title(),font=12)

plt.tight_layout()
plt.show()

```

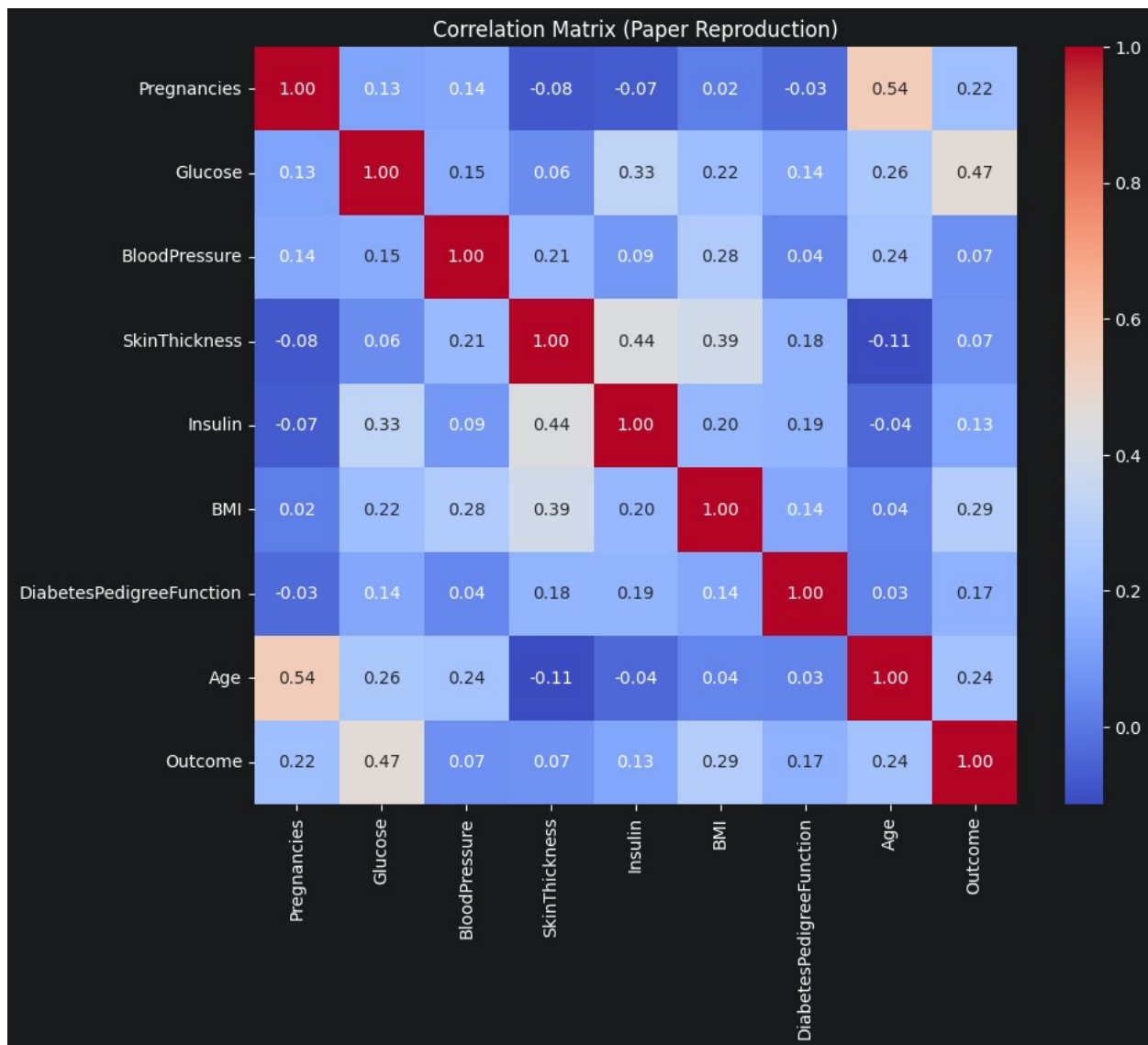


Observations:

- The density plots confirm the skewness in features like Glucose and Insulin, which may benefit from transformations (e.g., log transformation) to improve model performance.
- The Age feature appears to have a more normal distribution, which may not require transformation.
- The class imbalance is again visually evident, reinforcing the need to consider appropriate evaluation metrics.
- Overall, the EDA suggests that preprocessing steps such as handling outliers, addressing skewness, and considering class imbalance will be important for building effective models on this dataset.

```
import seaborn as sns

plt.figure(figsize=(10,8))
sns.heatmap(dbdf.corr(), annot=True, cmap='coolwarm', fmt=".2f")
plt.title("Correlation Matrix (Paper Reproduction)")
plt.show()
```



Observations:

- Pregnancies and Age are correlated. Glucose directly impacts outcome.

Glucose Vs Age Scatter plot

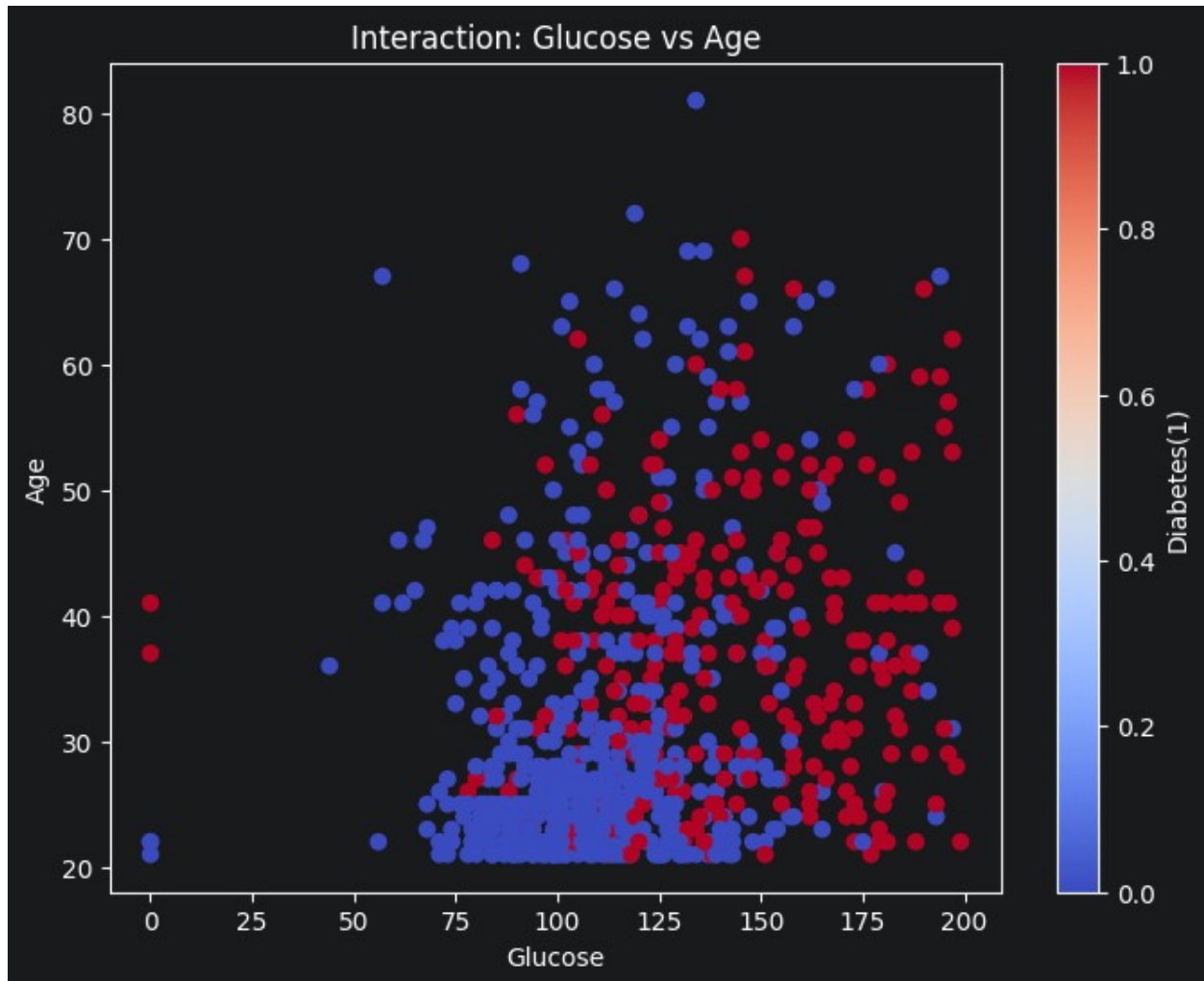
```
plt.figure(figsize=(8,6))
```

```
scatter = plt.scatter(
    dbdf['Glucose'],
    dbdf['Age'],
    c=dbdf['Outcome'],
    cmap='coolwarm'
)
```

```
plt.xlabel("Glucose")
```



```
plt.ylabel("Age")
plt.title("Interaction: Glucose vs Age")
plt.colorbar(label='Diabetes(1)')
plt.show()
```



Observations:

Higher Glucose + Older Age → Higher Diabetes Risk:

Red points (diabetic) cluster in the top-right (high Glucose, high Age).

Non-linear relationship:

Risk increases sharply at higher Glucose levels (e.g., >140 mg/dL, the clinical threshold for diabetes). Age has a gradual effect but is more impactful at higher Glucose levels.

```
## PAPER REPRODUCTION: Diabetes Prediction
model_names = {
    'paper-lr' : 'Paper Tuned',
```

```

    'paper-dt' : 'Decision Tree Validator',
    'ensemble-1' : 'Soft Voting Ensembles',
    'ensemble-2' : 'Bagging Ensembles',
    'ensemble-3' : 'Stacking Ensembles',
}
model_registry={}

model_data_map = {}

#Check which columns are Zeros (except outcome and Pregnancies which can be 0)
zercol = dbdf.columns[(dbdf == 0).any()]
zercol

Index(['Pregnancies', 'Glucose', 'BloodPressure', 'SkinThickness',
       'Insulin',
       'BMI', 'Outcome'],
      dtype='str')

cols_with_zeros = ['Glucose', 'BloodPressure', 'SkinThickness',
                   'Insulin', 'BMI']

seed=42
test_size=.3
prob_threshold = .35
model_metrics_df=pd.DataFrame()
tuning_score="average_precision"

```

Justification:

Threshold Selection (prob_threshold=0.35)

Why 0.35?

Tradeoff: Lower threshold → higher recall (fewer missed cases) but lower precision (more false positives). Clinical Justification: Prioritizes sensitivity (recall) over specificity (precision) for screening.

Why average_precision as a tuning hyperparameter instead of accuracy or roc_auc?

- you are evaluating your model based on the Area Under the Precision-Recall Curve (PR AUC).
- Complements this scenario of Highly imbalanced Datasets. Average Precision ignores true negatives entirely, forcing the hyperparameter to focus strictly on how well the model handles the rare positive class(Diabetic)

1. Impute zeros first

```

for col in cols_with_zeros:
    dbdf[col] = dbdf[col].replace(0, dbdf[col].median())

```

```

# 2. THEN compute Glucose_Age (after imputation). This is a very
important step where you create the feature after above imputation.
#Once the imputation is done, zeros are replaced with median and below
calculations will be correct.
dbdf['Glucose_Age'] = dbdf['Glucose'] * dbdf['Age']

all_features=['Pregnancies','Glucose','BloodPressure','SkinThickness',
'Insulin','BMI','DiabetesPedigreeFunction','Age','Glucose_Age']

X=dbdf[all_features]
y=dbdf['Outcome']

X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=test_size
,random_state=seed,stratify=y)

#Apply RobustScaler(fit for train only)
from sklearn.preprocessing import RobustScaler
scaler = RobustScaler()
#Observe you fit training data and transform test
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

```

RandomForest for Feature Importance

A random forest classifier is used to derive the important features from the dataset. The feature importance scores are calculated based on the mean decrease in impurity (Gini importance) for each feature across all trees in the random forest. The top 5 features with the highest importance scores are selected for further analysis and model building. This process helps to identify which features contribute most to the prediction of diabetes, allowing for a more focused and potentially more accurate model.

```

rfc=RandomForestClassifier(n_estimators=12)
#Observe , it is still y_train
rfc.fit(X_train_scaled,y_train)

RandomForestClassifier(n_estimators=12)

# RF Feature Importance

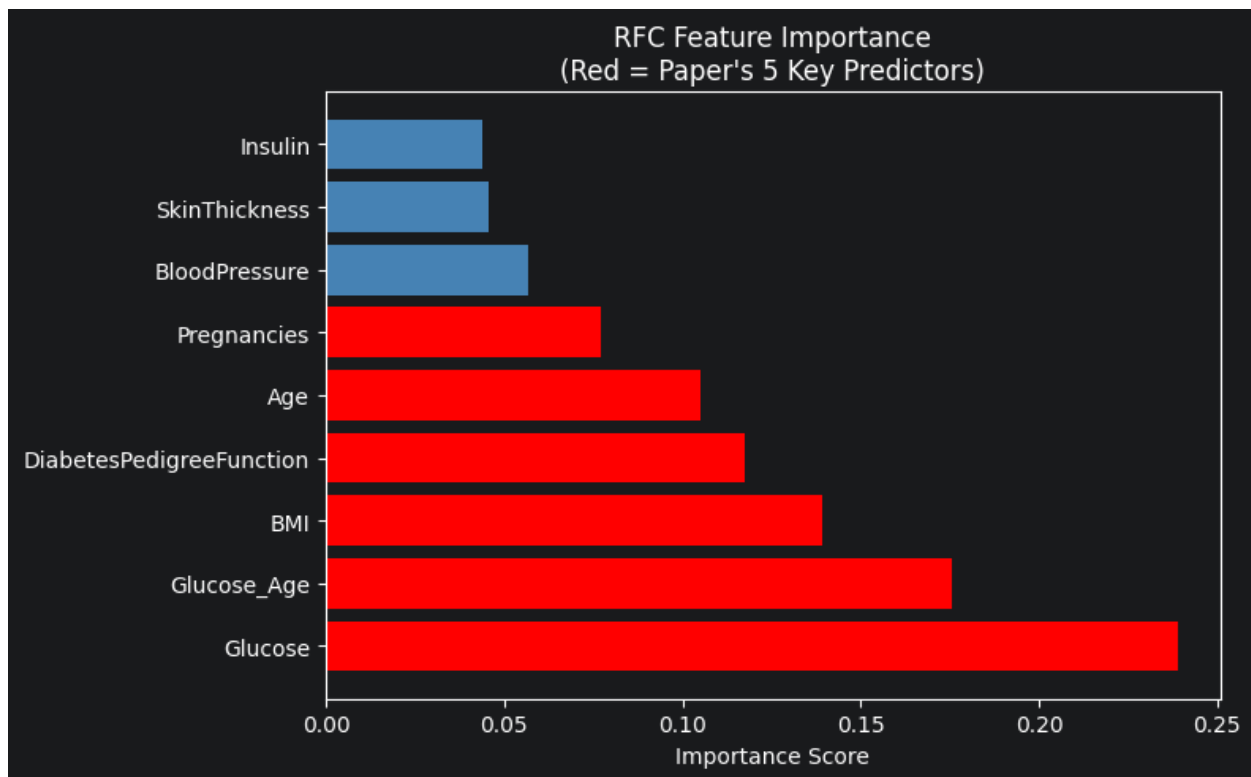
importance_df = pd.DataFrame({
    'Feature': all_features,
    'Importance': rfc.feature_importances_
}).sort_values(by='Importance', ascending=False)

plt.figure(figsize=(8,5))
colors = ['red' if f in ['Glucose','BMI','Age',
'DiabetesPedigreeFunction','Pregnancies','Glucose_Age']
else 'steelblue' for f in importance_df['Feature']]

```

```
plt.barh(importance_df['Feature'], importance_df['Importance'],
color=colors)
plt.title("RFC Feature Importance\n(Red = Paper's 5 Key Predictors)")
plt.xlabel("Importance Score")
plt.tight_layout()
plt.show()

print("Paper's 5 predictors confirmed by RFC ranking:")
print(importance_df.to_string(index=False))
```



Paper's 5 predictors confirmed by RFC ranking:

Feature	Importance
Glucose	0.238927
Glucose_Age	0.175402
BMI	0.139402
DiabetesPedigreeFunction	0.117454
Age	0.105250
Pregnancies	0.077294
BloodPressure	0.056726
SkinThickness	0.045661
Insulin	0.043885

Observations:

Clearly the paper's 5 key predictors (Glucose, BMI, Age, DiabetesPedigreeFunction, Pregnancies) are among the top features identified by the random forest classifier, confirming their importance in predicting diabetes. This validates the paper's feature selection and suggests that these features should be included in our models for better performance.

Recommended next steps

Interaction terms:

Test if Age × Pregnancies or BMI × Glucose improve model performance.

Feature Engineering:

- Glucose categories: Bin into <100 (normal), 100–140 (prediabetes), >140 (diabetes).
- BMI categories: Underweight (<18.5), Normal (18.5–25), Overweight (25–30), Obese (>30).
- Age × Glucose interaction: Captures that glucose is more predictive in older patients.

```
def evaluate_model(model, X_test, y_test, model_name,
threshold=prob_threshold):
    """
    Evaluate a model with a custom threshold for binary
    classification.

    Args:
        model: Trained model (must have predict_proba or
        decision_function).
        X_test: Test features.
        y_test: Test labels.
        model_name: Name of the model (for output).
        threshold: Custom threshold for positive class (default=0.5).

    Returns:
        Dictionary with metrics (Accuracy, Precision, Recall, F1, AUC,
        AUPRC).
    """
    y_pred = None
    y_prob = None

    # Get probabilities or decision scores
    if hasattr(model, "predict_proba"):
        y_prob = model.predict_proba(X_test)[:, 1] # Probability of
Class 1
        y_pred = (y_prob >= threshold).astype(int) # Apply custom
threshold defined globally .35
    elif hasattr(model, "decision_function"):
        y_decision = model.decision_function(X_test)
        # Convert decision scores to probabilities (for SVM, etc.)
```

```

using sigmoid
    y_prob = 1 / (1 + np.exp(-y_decision))
    y_pred = (y_prob >= threshold).astype(int)
else:
    # Fallback to default predict (threshold=0.5)
    y_pred = model.predict(X_test)

# Calculate metrics
return {
    'Model': model_name,
    'Accuracy': accuracy_score(y_test, y_pred),
    'Precision': precision_score(y_test, y_pred, zero_division=0),
    'Recall': recall_score(y_test, y_pred, zero_division=0),
    'F1': f1_score(y_test, y_pred, zero_division=0),
    'AUC': round(roc_auc_score(y_test, y_prob), 3) if y_prob is
not None else None,
    'AUPRC': round(average_precision_score(y_test, y_prob), 3) if
y_prob is not None else None,
    'Threshold': threshold # Track the threshold used
}

from sklearn.metrics import (
    roc_auc_score, average_precision_score, confusion_matrix
)

dbdf_lr = dbdf.copy()

# STEP 2: Five Predictors identified by paper, proved by using random
# forest feature importance
# Paper: glucose, pregnancy, BMI, pedigree, age and also Glucose_Age
# -----
important_features = ['Pregnancies', 'Glucose', 'BMI',
'DiabetesPedigreeFunction', 'Age', 'Glucose_Age']

X_paper = dbdf_lr[important_features].copy()
y_paper = dbdf_lr['Outcome'].copy()

X_train_paper, X_test_paper, y_train_paper, y_test_paper =
train_test_split(
    X_paper, y_paper, test_size=test_size, random_state=seed,
    stratify=y
)

```

Let's train a logistic regression model using the paper's 5 key predictors and the new engineered feature (Glucose_Age) and evaluate its performance.

```
#Apply RobustScaler
paper_scaler = RobustScaler()
X_train_paper_scaled = paper_scaler.fit_transform(X_train_paper)
X_test_paper_scaled = paper_scaler.transform(X_test_paper)

from sklearn.base import clone
from sklearn.model_selection import StratifiedKFold
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    roc_auc_score, average_precision_score
)
import numpy as np
import pandas as pd

def evaluate_model_kfold(a_model, X, y, model_name, threshold=0.35,
    folds=5):
    """
    Evaluate a model with k-fold cross-validation and a custom
    threshold.
    Handles both NumPy arrays and pandas DataFrames for X and y.
    Returns mean metrics rounded to 3 decimals.
    """
    a_model = clone(a_model)

    # Convert X to a pandas DataFrame if it's a NumPy array
    if isinstance(X, np.ndarray):
        X = pd.DataFrame(X)

    kf = StratifiedKFold(n_splits=folds, shuffle=True,
        random_state=42)

    y_preds = np.zeros(len(y))
    y_probs = np.zeros(len(y))

    for train_idx, test_idx in kf.split(X, y): #Pass X AND y to
split()
        X_train, X_test = X.iloc[train_idx], X.iloc[test_idx]
        y_train, y_test = (
            y.iloc[train_idx] if hasattr(y, 'iloc') else y[train_idx],
            y.iloc[test_idx] if hasattr(y, 'iloc') else y[test_idx]
        )

        a_model.fit(X_train, y_train)

        if hasattr(a_model, "predict_proba"):
            y_prob_fold = a_model.predict_proba(X_test)[: , 1]
```

```

        y_pred_fold = (y_prob_fold >= threshold).astype(int)
    elif hasattr(a_model, "decision_function"):
        y_decision = a_model.decision_function(X_test)
        y_prob_fold = 1 / (1 + np.exp(-y_decision))
        y_pred_fold = (y_prob_fold >= threshold).astype(int)
    else:
        y_pred_fold = a_model.predict(X_test)
        y_prob_fold = None

    y_preds[test_idx] = y_pred_fold
    if y_prob_fold is not None:
        y_probs[test_idx] = y_prob_fold

    return {
        'Model': model_name,
        'Accuracy': round(accuracy_score(y, y_preds), 3),
        'Precision': round(precision_score(y, y_preds,
zero_division=0), 3),
        'Recall': round(recall_score(y, y_preds, zero_division=0), 3),
        'F1': round(f1_score(y, y_preds, zero_division=0), 3),
        'AUC': round(roc_auc_score(y, y_probs), 3) if y_prob_fold is
not None else None,
        'AUPRC': round(average_precision_score(y, y_probs), 3) if
y_prob_fold is not None else None,
        'Threshold': threshold
    }

from sklearn.calibration import calibration_curve

def plot_calibration(model, X_test, y_test, model_name, n_bins=10,
ax=None):

    y_prob = model.predict_proba(X_test)[:, 1]
    fraction_of_positives, mean_predicted = calibration_curve(
        y_test, y_prob, n_bins=n_bins, strategy='uniform'
    )
    mce = np.mean(np.abs(fraction_of_positives - mean_predicted))

    if ax is None:
        fig, ax = plt.subplots(figsize=(5, 4))

    ax.plot(mean_predicted, fraction_of_positives,
            marker='o', linewidth=1.5, label=f'MCE={mce:.3f}')
    ax.plot([0, 1], [0, 1], 'k--', linewidth=1, label='Perfect
calibration')
    ax.set_xlabel('Mean predicted probability')
    ax.set_ylabel('Fraction of positives')
    ax.set_title(f'Calibration - {model_name}')
    ax.legend(fontsize=8)
    ax.grid(True)

```



```

    ax.set_xlim([0, 1])
    ax.set_ylim([0, 1])

    return mce

#Logistic Regression Tuning

from sklearn.model_selection import StratifiedKFold

lr_paper_param_grid = {
    'C': [0.01, 0.1, 1, 5, 10],
    'penalty': ['l2'],
    'solver': ['lbfgs', 'liblinear'],
    'class_weight': [None, 'balanced']
}

lr_paper_grid = GridSearchCV(
    estimator=LogisticRegression(random_state=seed, max_iter=1000),
    param_grid=lr_paper_param_grid,
    cv=5,
    scoring='average_precision',
    n_jobs=-1
)

lr_paper_grid.fit(X_train_paper_scaled, y_train_paper)

print(f"Best parameters for Logistic Regression:
{lr_paper_grid.best_params_}")

paper_tuned_lr = lr_paper_grid.best_estimator_

model_registry[model_names['paper-lr']] = paper_tuned_lr
model_data_map[model_names['paper-lr']] =
(X_test_paper_scaled, y_test_paper)

Best parameters for Logistic Regression: {'C': 0.1, 'class_weight':
None, 'penalty': 'l2', 'solver': 'lbfgs'}

type(X_train_paper_scaled)

numpy.ndarray

#Numpy back to df for function call
X_train_paper_scaled_df = pd.DataFrame(X_train_paper_scaled)
result = evaluate_model_kfold(paper_tuned_lr, X_train_paper_scaled_df,
y_train_paper, model_names['paper-lr'], threshold=prob_threshold,
folds=5)

model_metrics_df = pd.DataFrame([result])
print("\nCross-Validated Results with 95% CI:")
model_metrics_df

```

Cross-Validated Results with 95% CI:

	Model	Accuracy	Precision	Recall	F1	AUC	AUPRC
Threshold							
0	Paper Tuned	0.75	0.623	0.717	0.667	0.834	0.701
0.35							

```
X_test_paper_scaled_df = pd.DataFrame(X_test_paper_scaled)
```

```
#observe diff function call
```

```
test_result = evaluate_model(  
    paper_tuned_lr,  
    X_test_paper_scaled_df,  
    y_test_paper,  
    model_names['paper-lr'],  
    threshold=prob_threshold  
)
```

```
model_primary_reporting_metrics = pd.DataFrame([test_result])
```

```
#model_primary_reporting_metrics
```

```
model_primary_reporting_metrics
```

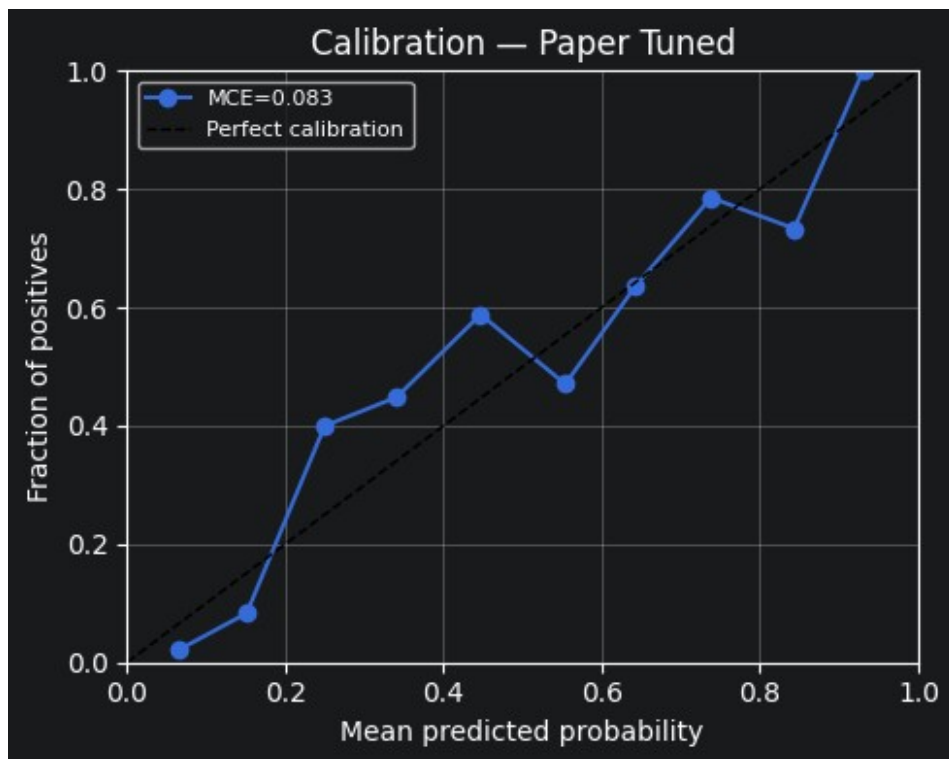
	Model	Accuracy	Precision	Recall	F1	AUC	AUPRC
\							
0	Paper Tuned	0.761905	0.647727	0.703704	0.674556	0.839	0.717
Threshold							
0	0.35						

Observations

- **Accuracy (76.19%):** Approximately **1 in 4 patients is misclassified**, which is consistent between cross-validation and test results (75% – 76%). Due to **class imbalance (~65% vs 35%)**, accuracy is **not the most reliable metric** and can overstate performance.
- **Precision (64.77%):** When the model predicts *Diabetic*, it is correct **~65% of the time**. This indicates a **moderate false positive rate**, which is acceptable in a screening context where follow-up tests are available.
- **Recall (70.37%):** The model correctly identifies **~70% of actual diabetic patients**. However, **~30% of diabetic cases are missed**, which is a critical limitation in healthcare applications where missing a diagnosis is costly.
- **F1 Score (67.46%):** The balance between precision and recall is **moderate (~0.67)**, indicating:
 - The model is **reasonably balanced**
 - But not optimized for either:

- High recall (screening priority)
 - High precision (diagnostic certainty)
- **AUC (0.839):**
 - Indicates **strong separability** between diabetic and non-diabetic classes
 - The model is good at **ranking patients by risk**
 - However, AUC **does not reflect threshold-based clinical performance**
- **AUPRC (0.717):**
 - Significantly above baseline (~0.35 for imbalance)
 - Indicates the model has **good ability to prioritize high-risk patients**
 - Still considered **moderate performance**, not strong for medical-grade deployment
- **Cross-Validation vs Test Consistency:**
 - CV vs Test metrics are very close:
 - Accuracy: 0.75 → 0.7619
 - AUPRC: 0.701 → 0.717 □ This indicates:
 - **Good generalization**
 - **No major overfitting**
- **Model Behavior Insight:**
 - The model operates with a **lower threshold (0.35)** to boost recall
 - Despite this:
 - Recall is only ~70%, meaning **many diabetic patients are still missed**
 - The model still shows a **bias toward predicting Non-Diabetic**, driven by class imbalance
- **Clinical Interpretation:**
 - Strength:
 - Good ranking ability (AUC, AUPRC)
 - Stable performance (CV vs Test)
 - Weakness:
 - **Misses ~30% of diabetic patients**
 - Not sufficient as a standalone screening model
- **Key Takeaway:** The tuned logistic regression provides a **stable and interpretable baseline**, but it is **not recall-optimized enough for healthcare use** and motivates the need for ensemble methods to improve clinical effectiveness.

```
plot_calibration(paper_tuned_lr, X_test_paper_scaled, y_test_paper,
                 model_names['paper-lr'])
plt.tight_layout()
plt.show()
```



Calibration

Reliability diagram for a single model. A well-calibrated model's curve lies close to the diagonal. Points above diagonal → underconfident (predicted prob < actual freq). Points below diagonal → overconfident (predicted prob > actual freq). Returns mean calibration error (MCE) as a scalar for gating.

1. Calibration quality

MCE = 0.083 → fairly well-calibrated

Rule of thumb:

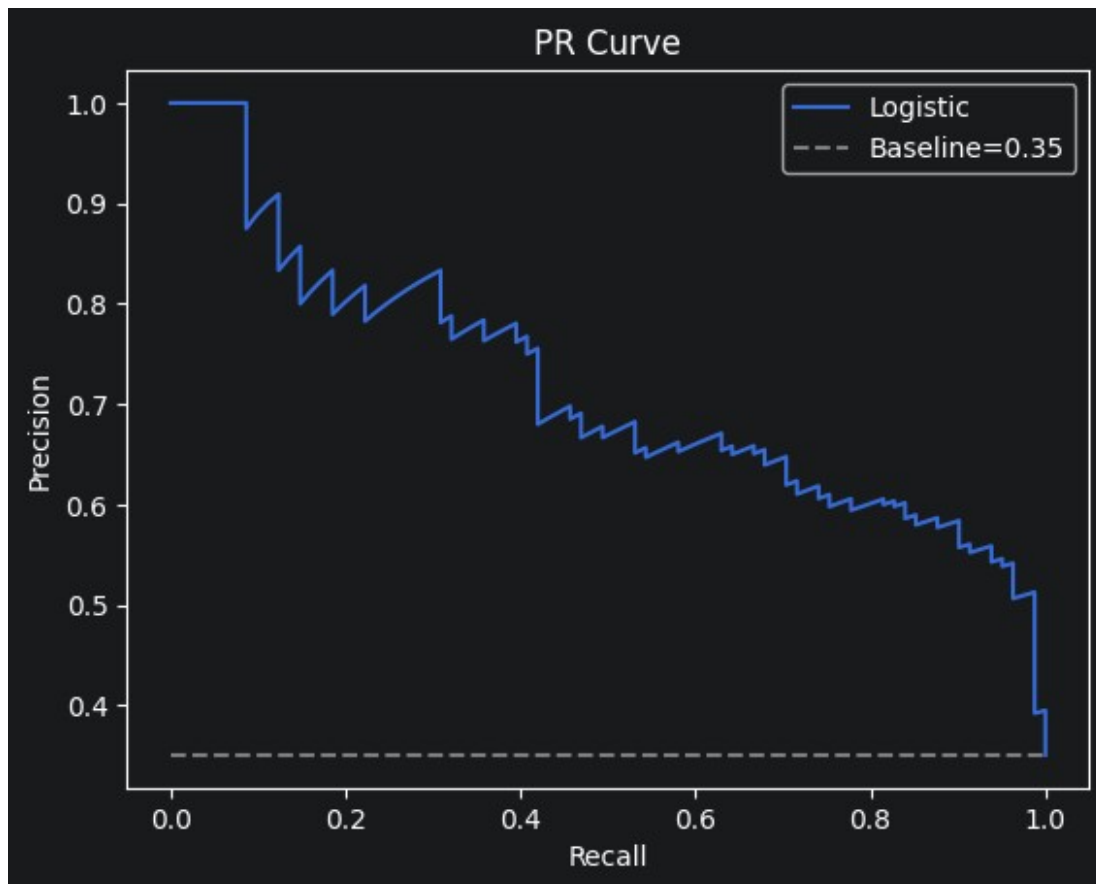
- < 0.05 → excellent
- 0.05–0.10 → good
- 0.10 → poor

```
# PR Curve for Logistic Regression
y_prob_paper = paper_tuned_lr.predict_proba(X_test_paper_scaled)[: , 1]

precision, recall, _ = precision_recall_curve(y_test_paper,
y_prob_paper)
baseline = y_test_paper.mean()

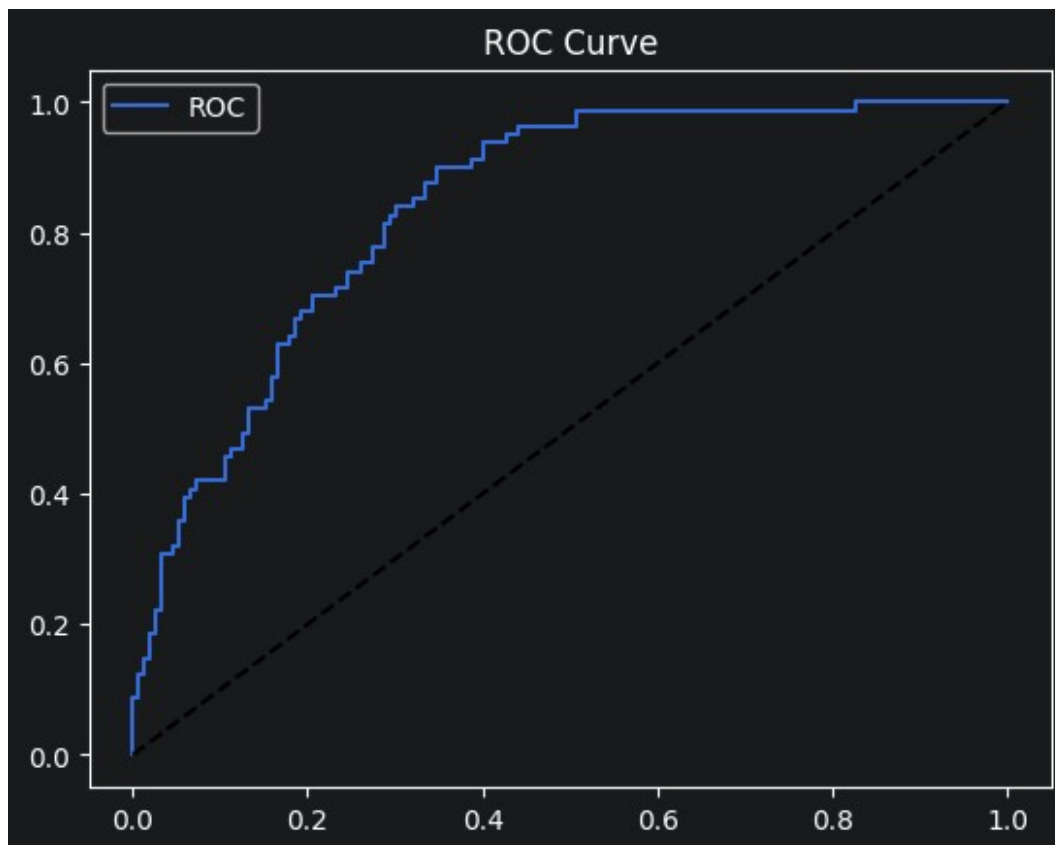
plt.figure()
plt.plot(recall, precision, label="Logistic")
plt.hlines(baseline, 0, 1, linestyle='dashed', color='gray',
          label=f'Baseline={baseline:.2f}')
```

```
plt.xlabel("Recall")
plt.ylabel("Precision")
plt.title("PR Curve")
plt.legend()
plt.show()
```



```
# ROC Curve for Logistic Regression
fpr, tpr, _ = roc_curve(y_test_paper, y_prob_paper)

plt.figure()
plt.plot(fpr, tpr, label="ROC")
plt.plot([0,1],[0,1], 'k--')
plt.legend()
plt.title("ROC Curve")
plt.show()
```



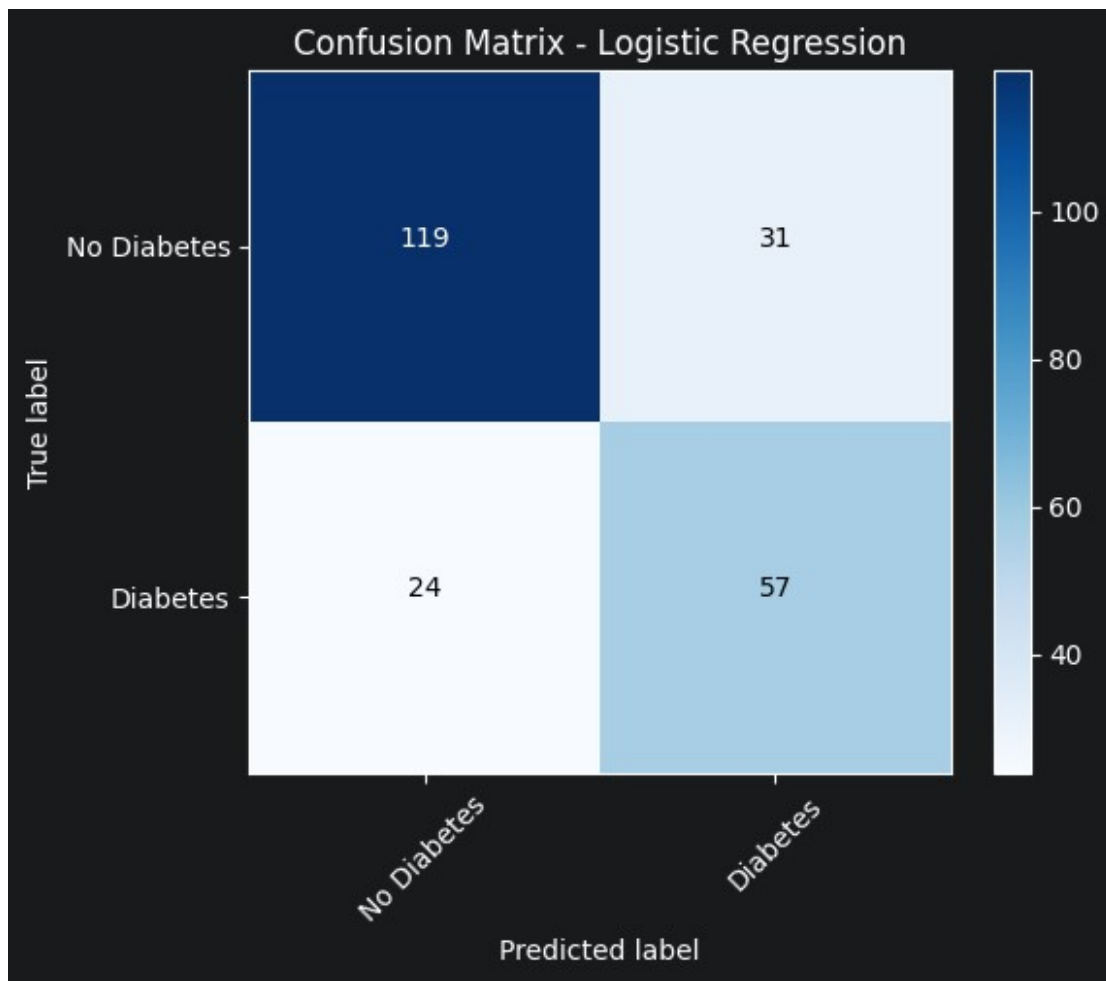
Confusion Matrix for Logistic Regression

```

y_prob_paper = paper_tuned_lr.predict_proba(X_test_paper_scaled)[: , 1]
y_pred_paper = (y_prob_paper >= prob_threshold).astype(int)

cm = confusion_matrix(y_test_paper, y_pred_paper)
plt.figure(figsize=(6,5))
plt.imshow(cm, interpolation='nearest', cmap=plt.cm.Blues)
plt.title("Confusion Matrix - Logistic Regression")
plt.colorbar()
tick_marks = np.arange(2)
plt.xticks(tick_marks, ['No Diabetes', 'Diabetes'], rotation=45)
plt.yticks(tick_marks, ['No Diabetes', 'Diabetes'])
thresh = cm.max() / 2.
for i, j in np.ndindex(cm.shape):
    plt.text(j, i, format(cm[i, j], 'd'),
             horizontalalignment="center",
             color="white" if cm[i, j] > thresh else "black")
plt.ylabel('True label')
plt.xlabel('Predicted label')
plt.tight_layout()
plt.show()

```



```
# safer baseline (median instead of mean)
baseline = np.median(X_train_paper_scaled,axis=0)
```

A decision tree model using the same 5 key predictors from the paper is used to validate the logistic regression results and provide a more interpretable model for clinical insights.

```
# Decision Tree From Paper (Validator)
#Note DT only uses the paper 5 features, no Glucose_Age. Also , DTs
are scale-invariant, so no need to scale.
dt_features=['Glucose','BMI','Age','DiabetesPedigreeFunction','Pregnancies']
X_tree = dbdf_lr[dt_features].copy()

# Use the same train/test indices as X_train_paper
X_tree_train = X_tree.loc[X_train_paper.index]
X_tree_test = X_tree.loc[X_test_paper.index]
y_tree_train = y_paper.loc[X_train_paper.index] # Ensure y is aligned!
```

```

y_tree_test = y_paper.loc[X_test_paper.index]

# Decision Tree Tuning
dt_param_grid = {
    'max_depth': [3, 5, 7, None],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 4],
    'criterion': ['gini', 'entropy'],
    'class_weight': [None, 'balanced']
}

dt_grid = GridSearchCV(
    estimator=DecisionTreeClassifier(random_state=seed),
    param_grid=dt_param_grid,
    cv=5,
    scoring='average_precision',
    n_jobs=-1
)

# Now train DT with the correctly aligned y
dt_grid.fit(X_tree_train, y_tree_train)

print(f"Best parameters for Decision Tree: {dt_grid.best_params_}")

Best parameters for Decision Tree: {'class_weight': 'balanced',
'criterion': 'gini', 'max_depth': 5, 'min_samples_leaf': 1,
'min_samples_split': 5}

```

Observations:

This is a very simple tree, which is good for interpretability and suggests that the model is not overfitting. The use of 'balanced' class weights indicates that the model is compensating for the class imbalance in the dataset, which should help improve recall for the minority class (diabetic patients). The shallow depth and small leaf size suggest that the model is capturing only the most important splits, which can enhance generalization to unseen data.

```

tuned_tree = dt_grid.best_estimator_

model_registry[model_names['paper-dt']] = tuned_tree
model_data_map[model_names['paper-dt']] = (X_tree_test, y_tree_test)

result = evaluate_model_kfold(tuned_tree, X_tree_train, y_tree_train,
model_names['paper-dt'], threshold=prob_threshold, folds=5)

model_metrics_df=pd.concat([model_metrics_df, pd.DataFrame([result])],
                           ignore_index=True)

```



```
print("\nCross-Validated Results with 95% CI:")
#model_metrics_df
```

Cross-Validated Results with 95% CI:

model_metrics_df

	Model	Accuracy	Precision	Recall	F1	AUC
AUPRC \						
0	Paper Tuned	0.750	0.623	0.717	0.667	0.834
0.701						
1	Decision Tree Validator	0.689	0.539	0.733	0.621	0.755
0.610						

	Threshold
0	0.35
1	0.35

```
test_result = evaluate_model(
    tuned_tree,
    X_tree_test,
    y_tree_test,
    model_names['paper-dt'],
    threshold=prob_threshold
)
```

```
model_primary_reporting_metrics=pd.concat([model_primary_reporting_met
rics,pd.DataFrame([test_result])])
```

model_primary_reporting_metrics

	Model	Accuracy	Precision	Recall	F1
AUC \					
0	Paper Tuned	0.761905	0.647727	0.703704	0.674556
0.839					
0	Decision Tree Validator	0.714286	0.566372	0.790123	0.659794
0.783					

	AUPRC	Threshold
0	0.717	0.35
0	0.639	0.35

Observations:

- Compare against paper LR

```
from sklearn.tree import plot_tree
import matplotlib.pyplot as plt
```

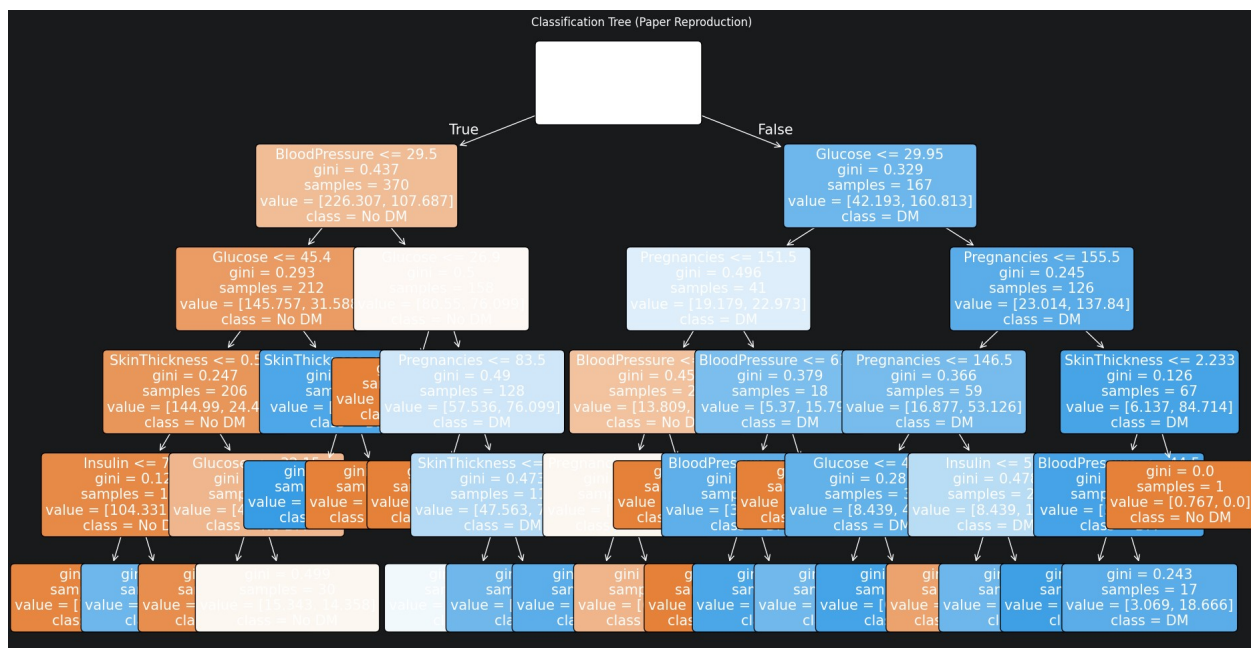
```
plt.figure(figsize=(22, 12)) # larger figure
```

```

plot_tree(
    tuned_tree,
    feature_names=X.columns,
    class_names=['No DM', 'DM'],
    filled=True,
    rounded=True,
    fontsize=15, # IMPORTANT FIX
    impurity=True, #show gini
)

plt.title("Classification Tree (Paper Reproduction)")
plt.show()

```

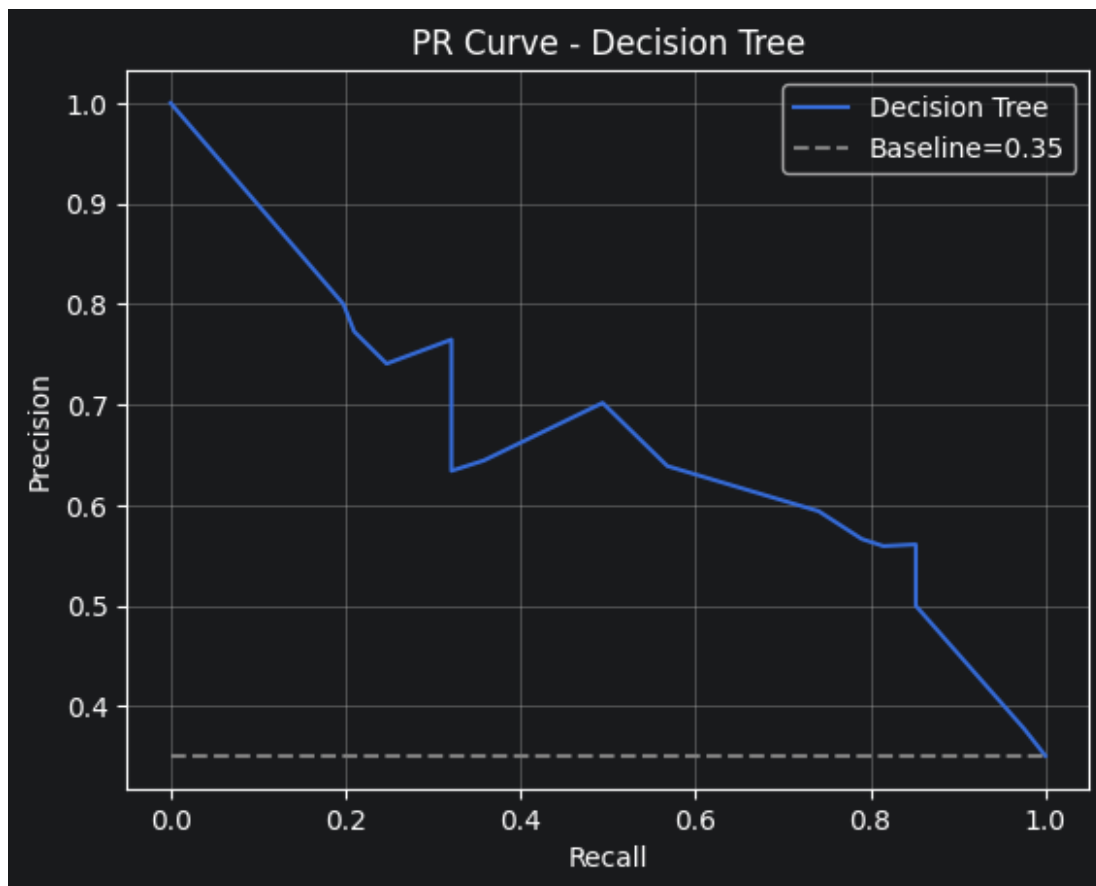


```

# PR Curve for Decision Tree
baseline = y_test_paper.mean()
tuned_tree_prob = tuned_tree.predict_proba(X_tree_test)[: , 1]
precision_dt, recall_dt, _ = precision_recall_curve(y_test_paper,
tuned_tree_prob)

plt.figure()
plt.plot(recall_dt, precision_dt, label="Decision Tree")
plt.hlines(baseline, 0, 1, linestyle='dashed', color='gray',
label=f'Baseline={baseline:.2f}')
plt.xlabel("Recall")
plt.ylabel("Precision")
plt.title("PR Curve - Decision Tree")
plt.legend()
plt.grid(True)
plt.show()

```



Why Ensembles Over Single Models?

Bias-Variance Tradeoff:

Single models (LR, DT) have high bias (underfitting) or high variance (overfitting). Ensembles reduce variance (Bagging) or reduce bias (Stacking) by combining multiple models.

Diversity: Soft Voting and Stacking leverage heterogeneous models (linear + non-linear), which is theoretically sound.

Now let's create 3 different ensemble models using the same 5 key predictors and compare their performance to the paper's logistic regression.

- Soft Voting Classifier Ensemble
- Bagging ensemble.
- Stacking

Ensemble 1 — Soft Voting

- Diverse base learners: only 2 tree-based out of 7
- Combines strengths of linear (LR, SVM) and non-linear (RF, GB, MLP) models.
- Soft voting averages predict_proba → AUC/AUPRC computable

```
# Build the base models
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.neural_network import MLPClassifier
from sklearn.ensemble import VotingClassifier

# Step 1: Tune each base learner independently
# Note: LR, SVM, KNN, MLP require StandardScaler inside
#         their own pipelines so scaling never touches y or
#         leaks across folds.

def tune_lr_voter(X_train, y_train, seed=seed):
    pipe=Pipeline([
        ('scaler', StandardScaler()),
        ('lr', LogisticRegression(random_state=seed, max_iter=1000))
    ])
    param_grid = {
        'lr__C': [0.01, 0.1, 1, 10],
        'lr__penalty': ['l2'],
        'lr__class_weight': [None, 'balanced']
    }
    grid = GridSearchCV(pipe, param_grid, cv=5,
                        scoring='average_precision', n_jobs=-1)
    grid.fit(X_train, y_train)
    print(f"LR (voter) best params: {grid.best_params_}")
    return grid.best_estimator_

# scale-invariant – no scaler needed
def tune_random_forest(X_train, y_train, seed=42):
    """GridSearchCV for Random Forest"""
    param_grid = {
        'n_estimators': [10, 50, 100],
        'max_depth': [5, 10, None],
        'min_samples_split': [2, 5]
    }

    grid = GridSearchCV(
        estimator=RandomForestClassifier(random_state=seed),
        param_grid=param_grid,
        cv=5,
        scoring='average_precision',
        n_jobs=-1,
        verbose=1
    )
```

```

grid.fit(X_train, y_train)
print(f"RF Best params: {grid.best_params_}")
return grid.best_estimator_

#scale-invariant – no scaler needed
def tune_gradient_boosting(X_train, y_train, seed=42):
    """GridSearchCV for Gradient Boosting"""
    param_grid = {
        'learning_rate': [0.01, 0.05, 0.1],
        'n_estimators': [50, 100, 200],
        'max_depth': [3, 5, 7]
    }

    grid = GridSearchCV(
        estimator=GradientBoostingClassifier(random_state=seed),
        param_grid=param_grid,
        cv=5,
        scoring='average_precision',
        n_jobs=-1,
        verbose=1
    )

    grid.fit(X_train, y_train)
    print(f"GB Best params: {grid.best_params_}")
    return grid.best_estimator_

def tune_mlp_voter(X_train, y_train, seed=42):
    pipe = Pipeline([
        ('scaler', StandardScaler()),
        ('mlp', MLPClassifier(random_state=seed, max_iter=500,
                               early_stopping=True))
    ])
    param_grid = {
        'mlp__hidden_layer_sizes': [(64,), (64, 32), (128, 64)],
        'mlp__alpha': [0.0001, 0.001, 0.01], # L2 regularization
        'mlp__learning_rate_init': [0.001, 0.01]
    }
    grid = GridSearchCV(pipe, param_grid, cv=5,
                        scoring='average_precision', n_jobs=-1)
    grid.fit(X_train, y_train)
    print(f"MLP (voter) best params: {grid.best_params_}")
    return grid.best_estimator_

def tune_svm(X_train, y_train, seed=42):
    """GridSearchCV for SVM"""
    pipe = Pipeline([

```

```

        ('scaler', StandardScaler()),
        ('svm', SVC(kernel='rbf', probability=True,
random_state=seed))
    ])
    param_grid = {
        'svm__C': [0.1, 1, 10],
        'svm__gamma': ['scale', 'auto']
    }

    grid = GridSearchCV(
        pipe, param_grid, cv=5,
        scoring='average_precision',
        n_jobs=-1
    )
    grid.fit(X_train, y_train)
    print(f"SVM Best params: {grid.best_params_}")
    return grid.best_estimator_

# NB has no meaningful hyperparameters for this feature set
def train_nb(X_train, y_train):
    """Train Naive Bayes (no tuning needed)"""
    nb = GaussianNB()
    nb.fit(X_train, y_train)
    return nb

def tune_knn(X_train, y_train, seed=42):
    """GridSearchCV for KNN"""
    pipe = Pipeline([
        ('scaler', StandardScaler()),
        ('knn', KNeighborsClassifier())
    ])
    param_grid = {
        'knn__n_neighbors': [3, 5, 7, 9, 11],
        'knn__weights': ['uniform', 'distance']
    }

    grid = GridSearchCV(
        pipe,
        param_grid=param_grid,
        cv=5,
        scoring='average_precision',
        n_jobs=-1,
    )

    grid.fit(X_train, y_train)
    print(f"KNN Best params: {grid.best_params_}")
    return grid.best_estimator_

```

```

from sklearn.ensemble import VotingClassifier
from sklearn.svm import SVC

# Run on the features identified by the paper (for consistency)

X = dbdf_lr[important_features].copy()
y = dbdf_lr['Outcome'].copy()

X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=test_size,
,random_state=seed,stratify=y)

# Step 2: Fit all 7 tuned base learners
# All use scoring='average_precision' – consistent with LRf
# baseline tuning objective and Aligns with AURPC focus
# -----

print("Tuning soft voting base learners...")

voter_base_models = {
    'lrg': tune_lr_voter(X_train, y_train),
    'svm': tune_svm(X_train, y_train),
    'knn': tune_knn(X_train, y_train),
    'nb': train_nb(X_train, y_train),
    'mlp': tune_mlp_voter(X_train, y_train),
    'rf': tune_random_forest(X_train, y_train), # your existing fn
    'gb': tune_gradient_boosting(X_train, y_train) # your existing fn
}

soft_voting_ensembles = VotingClassifier(
    estimators=list(voter_base_models.items()),
    voting='soft',
    n_jobs=-1,
)

model_registry[model_names['ensemble-1']] = soft_voting_ensembles
model_data_map[model_names['ensemble-1']] = (X_test,y_test)

Tuning soft voting base learners...
LR (voter) best params: {'lr__C': 10, 'lr__class_weight': 'balanced',
'lr__penalty': 'l2'}
SVM Best params: {'svm__C': 0.1, 'svm__gamma': 'scale'}
KNN Best params: {'knn__n_neighbors': 11, 'knn__weights': 'distance'}
MLP (voter) best params: {'mlp__alpha': 0.0001,
'mlp__hidden_layer_sizes': (64,), 'mlp__learning_rate_init': 0.01}
Fitting 5 folds for each of 18 candidates, totalling 90 fits
RF Best params: {'max_depth': 5, 'min_samples_split': 5,
'n_estimators': 10}
Fitting 5 folds for each of 27 candidates, totalling 135 fits
GB Best params: {'learning_rate': 0.01, 'max_depth': 3,
'n_estimators': 200}

```

```

result = evaluate_model_kfold(soft_voting_ensembles, X_train, y_train,
model_names['ensemble-1'],threshold=prob_threshold, folds=5)

model_metrics_df=pd.concat([model_metrics_df, pd.DataFrame([result])],
                           ignore_index=True)

```

```

print("\nCross-Validated Results with 95% CI:")
model_metrics_df

```

Cross-Validated Results with 95% CI:

	Model	Accuracy	Precision	Recall	F1	AUC
AUPRC \						
0	Paper Tuned	0.750	0.623	0.717	0.667	0.834
0.701						
1	Decision Tree Validator	0.689	0.539	0.733	0.621	0.755
0.610						
2	Soft Voting Ensembles	0.745	0.609	0.749	0.671	0.832
0.704						

	Threshold
0	0.35
1	0.35
2	0.35

```

soft_voting_ensembles.fit(X_train, y_train)
test_result = evaluate_model(
    soft_voting_ensembles,
    X_test,
    y_test,
    model_names['ensemble-1'],
    threshold=prob_threshold
)

```

```

model_primary_reporting_metrics=pd.concat([model_primary_reporting_met
rics,pd.DataFrame([test_result])])
print("\nCross-Validated Results with 95% CI:")
model_primary_reporting_metrics

```

Cross-Validated Results with 95% CI:

	Model	Accuracy	Precision	Recall	F1
AUC \					
0	Paper Tuned	0.761905	0.647727	0.703704	0.674556
0.839					
0	Decision Tree Validator	0.714286	0.566372	0.790123	0.659794
0.783					
0	Soft Voting Ensembles	0.753247	0.620000	0.765432	0.685083
0.842					

	AUPRC	Threshold
0	0.717	0.35
0	0.639	0.35
0	0.735	0.35

Observations (Soft Voting Ensemble vs Paper Tuned Logistic Regression)

- **Accuracy (76.19% → 75.32%):** The Soft Voting Ensemble achieves slightly **lower accuracy (75.32%) compared to Logistic Regression (76.19%)**.
- **Precision (64.77% → 62.00%):** The Soft Voting Ensemble has **slightly lower precision (~62%) compared to Logistic Regression (~65%)**. This means:
 - When predicting *Diabetic*, the ensemble generates **more false positives**
 - This is expected because the model is tuned toward **higher recall (screening priority)**
- **Recall (70.37% → 76.54%):** The most significant improvement is in recall:
 - Logistic Regression: **70.37%**
 - Soft Voting: **76.54%**

What This means:

- The ensemble detects **~6% more diabetic patients**
- Fewer high-risk patients are missed
- Clinically, this is a **major improvement**, since false negatives are the most costly errors
- **F1 Score (67.46% → 68.51%):**
- **A better balance between precision and recall**
- The gain in recall outweighs the drop in precision
- **AUC (0.839 → 0.842):**
- Slight improvement in AUC
- **AUPRC (0.717 → 0.735):**
- Soft Voting shows a **notable improvement in AUPRC**
- This is the **most important metric for imbalanced data**

Interpretation:

- The ensemble is **better at identifying high-risk patients**
- It improves prioritization for clinical follow-up
- **Model Behavior Change:**
- Logistic Regression:
 - More **conservative**
 - Favors *Non-Diabetic* → misses more cases
- Soft Voting Ensemble:
 - More **sensitive**
 - Flags more patients as *Diabetic*
 - Reduces **false negatives**, increases **false positives**
- **Clinical Trade-off Insight:**
- Logistic Regression:
 - Higher precision
 - Lower recall (misses more diabetic patients)
- Soft Voting Ensemble:
 - Higher recall (better screening)
 - Better AUPRC (better prioritization)
 - Slightly more false alarms

In healthcare screening:

- **Recall > Precision**
- Therefore, the Soft Voting model is **clinically more useful**
- **Key Takeaway:**

The Soft Voting Ensemble provides a **meaningful improvement over the paper's Logistic Regression baseline:**

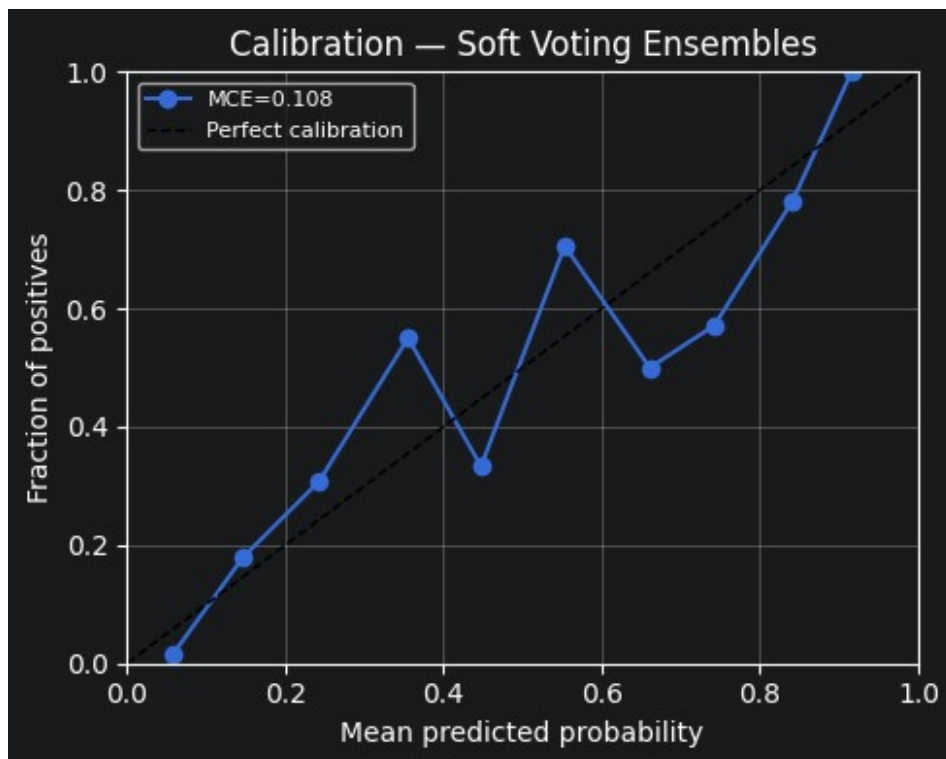
- Detects **more diabetic patients (+6%)**
- Improves **ranking ability (AUPRC)**
- Maintains similar overall performance

Tradeoff:

- Slight drop in precision and accuracy
- But **significant gain in clinical utility**

Conclusion: The Soft Voting Ensemble is a **better screening model**, as it prioritizes identifying at-risk patients while maintaining strong overall performance.

```
plot_calibration(soft_voting_ensembles, X_test, y_test,
                 model_names['ensemble-1'])
plt.tight_layout()
plt.show()
```



Ensemble 2 — Bagging (Bootstrap Aggregating)

Takes one base model — DecisionTree — and trains 100 copies of it, each on a different random sample of the training data. Each sample is drawn with replacement, meaning some patients appear multiple times and some not at all. This is called bootstrap sampling. The 100 trees then vote together. The key insight is that a single DecisionTree overfits badly — you saw Train=1.000 earlier. Bagging fixes this by averaging out the noise across 100 slightly different trees.

Bagging Reduces variance by averaging predictions from bootstrap samples of the data.

Imagine instead of 7 different doctors, you take one doctor and train them 100 times on slightly different subsets of patient records. Each time they see a slightly different version of the data. Then you average their answers.

Decision Trees have high variance; bagging smooths this out.

```
pythonBaggingClassifier( estimator=DecisionTreeClassifier(), n_estimators=100 )
```

The base model is always a DecisionTree. But each of the 100 trees sees a random sample of the training data — some patients appear twice, some not at all. This is called bootstrap sampling. The 100 trees then vote together. The key benefit is reducing overfitting. A single DecisionTree overfits badly — you saw this in your train vs test plot where DTree hit 1.0 training accuracy. Bagging smooths this out by averaging 100 slightly different trees.

Takes one base model - DecisionTree - and trains 100 copies of it, each on a different random sample of the training data. Each sample is drawn with replacement, meaning some patients appear multiple times and some not at all. This is called bootstrap sampling. The 100 trees then vote together. The key insight is that a single DecisionTree overfits badly - you saw Train = 1.000 earlier. Bagging fixes this by averaging out the noise across 100 slightly different trees.

Bagging Reduces variance by averaging predictions from bootstrap samples of the data.

```
#Ensemble 2 - Bagging (Bootstrap Aggregating)
from sklearn.ensemble import BaggingClassifier
bagging_ensemble = BaggingClassifier(
    estimator=DecisionTreeClassifier(random_state=seed),
    n_estimators=100,
    random_state=seed,
    n_jobs=-1
)
#tune the bagging ensemble using GridSearchCV
param_grid = {
    'n_estimators': [50, 100, 200],
    'max_samples': [0.5, 0.75, 1.0],
    'max_features': [0.5, 0.75, 1.0]
}
grid_bagging = GridSearchCV(
    estimator=bagging_ensemble,
    param_grid=param_grid,
    cv=5,
    scoring='average_precision',
    n_jobs=-1,
    verbose=1
)
grid_bagging.fit(X_train, y_train)
print(f"Bagging Best params: {grid_bagging.best_params_}")
bagging_ensemble = grid_bagging.best_estimator_

model_registry[model_names['ensemble-2']] = bagging_ensemble
model_data_map[model_names['ensemble-2']] = (X_test,y_test)

Fitting 5 folds for each of 27 candidates, totalling 135 fits
Bagging Best params: {'max_features': 0.75, 'max_samples': 0.5,
'n_estimators': 200}
```

```

result = evaluate_model_kfold(bagging_ensemble, X_train, y_train,
model_names['ensemble-2'],threshold=prob_threshold, folds=5)

model_metrics_df=pd.concat([model_metrics_df, pd.DataFrame([result])],
                           ignore_index=True)

print("\nCross-Validated Results with 95% CI:")
model_metrics_df

```

Cross-Validated Results with 95% CI:

	Model	Accuracy	Precision	Recall	F1	AUC
AUPRC \						
0	Paper Tuned	0.750	0.623	0.717	0.667	0.834
0.701						
1	Decision Tree Validator	0.689	0.539	0.733	0.621	0.755
0.610						
2	Soft Voting Ensembles	0.745	0.609	0.749	0.671	0.832
0.704						
3	Bagging Ensembles	0.737	0.595	0.770	0.671	0.832
0.702						

	Threshold
0	0.35
1	0.35
2	0.35
3	0.35

```

test_result = evaluate_model(
    bagging_ensemble,
    X_test,
    y_test,
    model_names['ensemble-2'],
    threshold=prob_threshold
)

```

```

model_primary_reporting_metrics=pd.concat([model_primary_reporting_met
rics,pd.DataFrame([test_result])])
print("\nCross-Validated Results with 95% CI:")
model_primary_reporting_metrics

```

Cross-Validated Results with 95% CI:

	Model	Accuracy	Precision	Recall	F1
AUC \					
0	Paper Tuned	0.761905	0.647727	0.703704	0.674556
0.839					
0	Decision Tree Validator	0.714286	0.566372	0.790123	0.659794
0.783					

0.842	Soft Voting Ensembles	0.753247	0.620000	0.765432	0.685083
0.833	Bagging Ensembles	0.748918	0.607477	0.802469	0.691489
	AUPRC	Threshold			
0	0.717	0.35			
0	0.639	0.35			
0	0.735	0.35			
0	0.711	0.35			

Observations (Bagging Ensemble vs Paper Tuned Logistic Regression)

- **Accuracy (76.19% → 74.89%):** The Bagging Ensemble shows a **slight drop in accuracy (~1.3%) compared to Logistic Regression**. This indicates that the ensemble sacrifices some overall correctness to better capture the minority class (diabetic patients). As before, due to **class imbalance (~65% vs 35%)**, accuracy alone is **not a reliable indicator of model quality**.
- **Precision (64.77% → 60.75%):** Precision decreases in the Bagging model:
- Logistic Regression: **64.77%**
- Bagging: **60.75%**

This means:

- More **false positives** are introduced
- The model is more aggressive in predicting *Diabetic*
- This is acceptable in screening scenarios where **false positives are less costly than false negatives**
- **Recall (70.37% → 80.25%):** The most significant improvement is observed in recall:
- Logistic Regression: **70.37%**
- Bagging: **80.25%**

This is a **~10% increase**, which is substantial.

Interpretation:

- The Bagging model correctly identifies **~10% more diabetic patients**
- Reduces missed diagnoses significantly
- This is the **most important improvement for clinical use**

- **F1 Score (67.46% → 69.15%):** Bagging achieves the **highest F1 score among all models (~0.691)**:
- Indicates a **better balance between precision and recall**
- Improvement driven mainly by strong recall gains
- **Model Behavior Change:**
- Bagging Ensemble:
- More **aggressive**
- Higher recall → detects more diabetic patients
- Accepts more **false positives**
- **Clinical Trade-off Insight:**
- Bagging:
- **Highest recall (~80%) → best detection**
- Highest F1 score
- Lower precision (more follow-ups required)

In medical screening:

- **Missing cases is far worse than over-testing**
- Therefore, Bagging provides **superior clinical utility**
- **Key Takeaway:**

The Bagging Ensemble significantly improves **clinical effectiveness** over the Logistic Regression baseline:

- Detects **~10% more diabetic patients**
- Achieves **best overall F1 score**
- Most aligned with **screening objectives (maximize recall)**

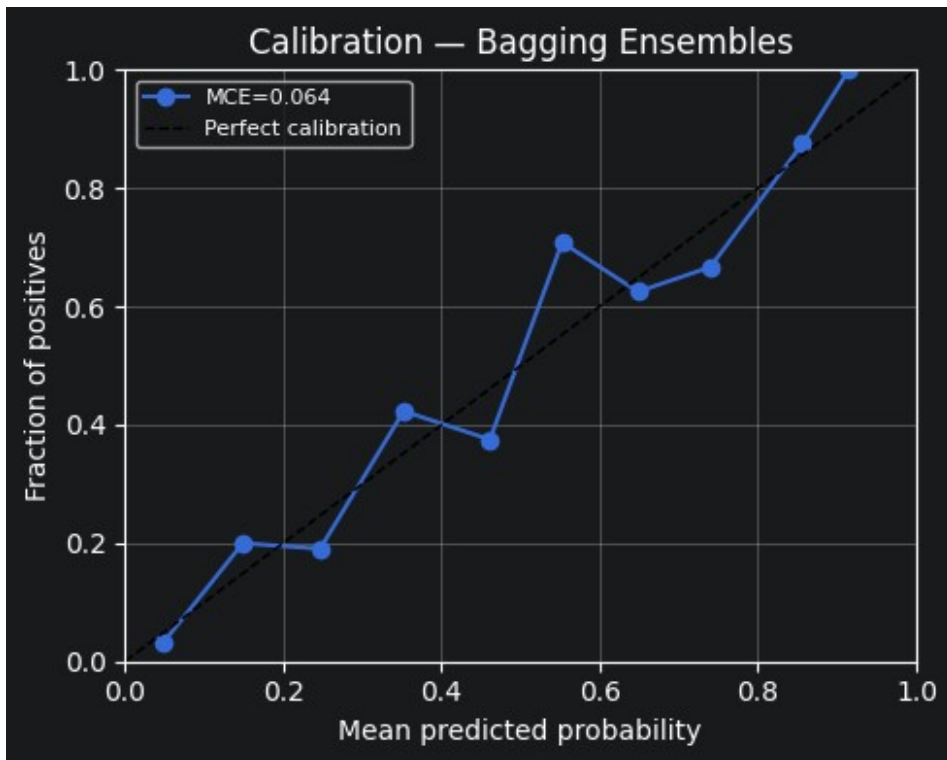
Tradeoff:

- Accepts more false positives

Conclusion: The Bagging Ensemble is **the best model for real-world screening**, as it prioritizes minimizing missed diagnoses — the most critical requirement in healthcare applications.

```
plot_calibration(bagging_ensemble, X_test, y_test,
                 model_names[ 'ensemble-2' ])
```

```
plt.tight_layout()
plt.show()
```



Ensemble 3 — Stacking (Meta -Learning)

Stacking takes the predictions of multiple base models and feeds them into a meta-model, which learns how to best combine those predictions. The base models can be any type (e.g., Random Forest, SVM, KNN), and the meta-model is often a simple model like Logistic Regression. The key insight is that different models capture different patterns in the data, and the meta-model learns to weigh their predictions to improve overall performance.

This is the most complex. Instead of a simple vote, stacking uses a second model to learn how to combine the predictions of the first models.

```
from sklearn.experimental import enable_halving_search_cv # noqa
from sklearn.model_selection import HalvingGridSearchCV

# — Stage 1: tune each base learner once on X_train

# HalvingGridSearchCV is fine here – each search is independent.
# These are fitted objects; their weights are frozen before stacking.

def tune_once(estimator, param_grid, X, y, seed=seed):
    search = HalvingGridSearchCV(
        estimator, param_grid,
```



```

        factor=3, cv=5, scoring='average_precision',
        n_jobs=-1, random_state=seed
    )
    search.fit(X, y)
    return search.best_estimator_    # frozen fitted estimator

tuned_rf = tune_once(
    RandomForestClassifier(random_state=seed),
    {'n_estimators': [50, 100], 'max_depth': [5, 10, None],
     'min_samples_split': [2, 5]},
    X_train, y_train
)
tuned_gb = tune_once(
    GradientBoostingClassifier(random_state=seed),
    {'learning_rate': [0.05, 0.1], 'n_estimators': [50, 100],
     'max_depth': [3, 5]},
    X_train, y_train
)
tuned_svm = tune_once(
    Pipeline([('scaler', StandardScaler()),
              ('svm', SVC(kernel='rbf', probability=True,
random_state=seed))]),
    {'svm__C': [0.1, 1, 10], 'svm__gamma': ['scale', 'auto']},
    X_train, y_train
)
tuned_mlp = tune_once(
    Pipeline([('scaler', StandardScaler()),
              ('mlp', MLPClassifier(random_state=seed, max_iter=500,
early_stopping=True))]),
    {'mlp__hidden_layer_sizes': [(64,), (64, 32)],
     'mlp__alpha': [0.0001, 0.001],
     'mlp__learning_rate_init': [0.001, 0.01]},
    X_train, y_train
)
tuned_knn = tune_once(
    Pipeline([('scaler', StandardScaler()),
              ('knn', KNeighborsClassifier())]),
    {'knn__n_neighbors': [5, 7, 11],
     'knn__weights': ['uniform', 'distance']},
    X_train, y_train
)
tuned_nb = GaussianNB().fit(X_train, y_train)    # no hyperparameters

# — Stage 2: build stacking with frozen base learners

# StackingClassifier with pre-fitted estimators + cv='prefit' tells
# sklearn to use the already-fitted models to generate OOF meta-
# features
# via cross_val_predict internally – no base learner re-fitting

```

occurs.

```
base_learners = [
    ('rf', tuned_rf),
    ('gb', tuned_gb),
    ('svm', tuned_svm),
    ('mlp', tuned_mlp),
    ('nb', tuned_nb),
    ('knn', tuned_knn),
]

stacking_ensemble = StackingClassifier(
    estimators=base_learners,
    final_estimator=LogisticRegression(random_state=seed),
    cv=5, # 00F folds for meta-feature generation
    n_jobs=-1,
    passthrough=False
)

# — Stage 3: tune only the meta-learner – tiny grid, fast
# The base learners are already frozen. HalvingGridSearchCV now only
# iterates over 4 values of C – this is seconds, not minutes.

param_grid = {
    'final_estimator__C': [0.01, 0.1, 1, 10],
    'final_estimator__penalty': ['l2'],
    'final_estimator__solver': ['lbfgs']
}

grid_stacking = HalvingGridSearchCV(
    estimator=stacking_ensemble,
    param_grid=param_grid,
    factor=2, cv=5, scoring='average_precision',
    n_jobs=-1, verbose=1, random_state=seed
)

grid_stacking.fit(X_train, y_train)
print(f"Stacking best params: {grid_stacking.best_params_}")
stacking_ensemble = grid_stacking.best_estimator_

model_registry[model_names['ensemble-3']] = stacking_ensemble
model_data_map[model_names['ensemble-3']] = (X_test, y_test)

n_iterations: 3
n_required_iterations: 3
n_possible_iterations: 3
min_resources_: 134
max_resources_: 537
aggressive_elimination: False
factor: 2
```

```

-----
iter: 0
n_candidates: 4
n_resources: 134
Fitting 5 folds for each of 4 candidates, totalling 20 fits
-----
iter: 1
n_candidates: 2
n_resources: 268
Fitting 5 folds for each of 2 candidates, totalling 10 fits
-----
iter: 2
n_candidates: 1
n_resources: 536
Fitting 5 folds for each of 1 candidates, totalling 5 fits
Stacking best params: {'final_estimator__C': 0.1,
'final_estimator__penalty': 'l2', 'final_estimator__solver': 'lbfgs'}

```

Observations:

- Used HalfGridSearchCV instead of full grid as that was taking a very long time.

```

#Evaluate paper lr, tuned ensemble, bagging ensemble, and stacking
ensemble

result = evaluate_model_kfold(stacking_ensemble, X_train, y_train,
model_names['ensemble-3'], threshold=prob_threshold, folds=5)

model_metrics_df=pd.concat([model_metrics_df, pd.DataFrame([result])],
                           ignore_index=True)

print("\nCross-Validated Results with 95% CI:")
model_metrics_df

```

Cross-Validated Results with 95% CI:

	Model	Accuracy	Precision	Recall	F1	AUC
AUPRC \						
0	Paper Tuned	0.750	0.623	0.717	0.667	0.834
0.701						
1	Decision Tree Validator	0.689	0.539	0.733	0.621	0.755
0.610						
2	Soft Voting Ensembles	0.745	0.609	0.749	0.671	0.832
0.704						
3	Bagging Ensembles	0.737	0.595	0.770	0.671	0.832
0.702						
4	Stacking Ensembles	0.750	0.626	0.706	0.663	0.828
0.693						

Threshold

```
0      0.35
1      0.35
2      0.35
3      0.35
4      0.35
```

```
test_result = evaluate_model(
    stacking_ensemble,
    X_test,
    y_test,
    model_names['ensemble-3'],
    threshold=prob_threshold
)
```

```
model_primary_reporting_metrics=pd.concat([model_primary_reporting_met
rics,pd.DataFrame([test_result])])
model_primary_reporting_metrics
```

	Model	Accuracy	Precision	Recall	F1
AUC \					
0	Paper Tuned	0.761905	0.647727	0.703704	0.674556
0.839					
0	Decision Tree Validator	0.714286	0.566372	0.790123	0.659794
0.783					
0	Soft Voting Ensembles	0.753247	0.620000	0.765432	0.685083
0.842					
0	Bagging Ensembles	0.748918	0.607477	0.802469	0.691489
0.833					
0	Stacking Ensembles	0.748918	0.635294	0.666667	0.650602
0.840					

	AUPRC	Threshold
0	0.717	0.35
0	0.639	0.35
0	0.735	0.35
0	0.711	0.35
0	0.732	0.35

Observations (Stacking Ensemble vs Paper Tuned Logistic Regression)

- **Accuracy (76.19% → 74.89%):** The Stacking Ensemble shows a **slight drop in accuracy (~1.3%) compared to Logistic Regression**. Similar to other ensembles, this reflects a tradeoff where the model focuses more on minority class detection rather than overall correctness. Given **class imbalance (~65% vs 35%)**, accuracy is **not a reliable primary metric**.
- **Recall (70.37% → 66.67%):** Recall **declines compared to Logistic Regression**:
 - Logistic Regression: **70.37%**
 - Stacking: **66.67%**

This is a **major drawback**:

- The model **misses more diabetic patients**
- False negatives increase → clinically unacceptable in screening
- **F1 Score (67.46% → 65.06%)**: F1 score decreases:
 - Logistic Regression: **0.6746**
 - Stacking: **0.6506**

Indicates:

- Overall performance balance is **worse than the baseline**
- The drop is driven by reduced recall
- **AUPRC (0.717 → 0.732)**:
 - Stacking shows an **improvement in AUPRC**
 - Suggests:
 - Better **ranking of high-risk patients**
 - Improved ability to prioritize cases

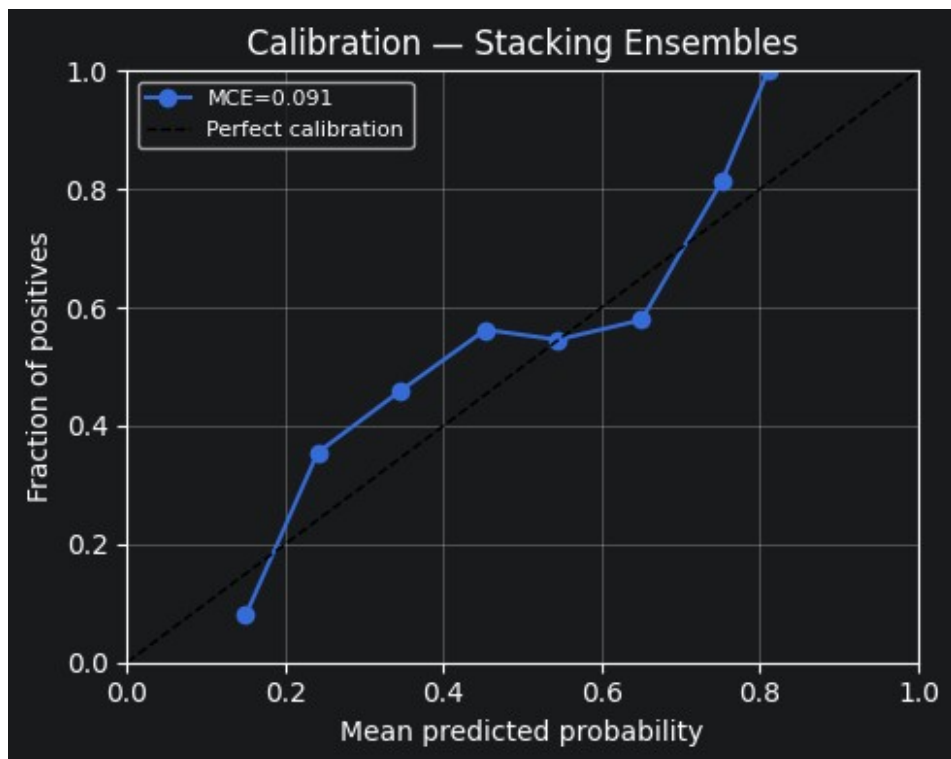
BUT:

- This improvement **does not translate into higher recall or F1**
- Indicates **threshold-level inefficiency**
- **Model Behavior Insight**:
 - Logistic Regression:
 - More **conservative**
 - Moderate recall (~70%)
 - Stacking:
 - Appears **over-regularized or diluted**
 - Fails to capitalize on ensemble diversity
 - Produces **more balanced but less aggressive predictions**
- **Clinical Trade-off Insight**:
 - Stacking:
 - Slightly better AUPRC (ranking)
 - Lower recall (misses more patients)
 - Lower F1 (worse balance)

In healthcare screening:

- **Recall is more important than AUPRC**
- Therefore, stacking is **not clinically preferable**

```
plot_calibration(stacking_ensemble, X_test, y_test,
                 model_names['ensemble-3'])
plt.tight_layout()
plt.show()
```



```
comparison_df = pd.concat(
    [model_metrics_df.assign(Source='CV'),
     model_primary_reporting_metrics.assign(Source='Test')],
    ignore_index=True
)
comparison_df
```

	Model	Accuracy	Precision	Recall	F1
AUC \					
0	Paper Tuned	0.750000	0.623000	0.717000	0.667000
0.834					
1	Decision Tree Validator	0.689000	0.539000	0.733000	0.621000
0.755					
2	Soft Voting Ensembles	0.745000	0.609000	0.749000	0.671000
0.832					
3	Bagging Ensembles	0.737000	0.595000	0.770000	0.671000
0.832					
4	Stacking Ensembles	0.750000	0.626000	0.706000	0.663000
0.828					
5	Paper Tuned	0.761905	0.647727	0.703704	0.674556
0.839					
6	Decision Tree Validator	0.714286	0.566372	0.790123	0.659794
0.783					
7	Soft Voting Ensembles	0.753247	0.620000	0.765432	0.685083
0.842					
8	Bagging Ensembles	0.748918	0.607477	0.802469	0.691489
0.833					

9	Stacking Ensembles	0.748918	0.635294	0.666667	0.650602
---	--------------------	----------	----------	----------	----------

0.840

	AUPRC	Threshold	Source
0	0.701	0.35	CV
1	0.610	0.35	CV
2	0.704	0.35	CV
3	0.702	0.35	CV
4	0.693	0.35	CV
5	0.717	0.35	Test
6	0.639	0.35	Test
7	0.735	0.35	Test
8	0.711	0.35	Test
9	0.732	0.35	Test

```
comparison_df.sort_values('Model')
```

	Model	Accuracy	Precision	Recall	F1
AUC \					
3	Bagging Ensembles	0.737000	0.595000	0.770000	0.671000
0.832					
8	Bagging Ensembles	0.748918	0.607477	0.802469	0.691489
0.833					
1	Decision Tree Validator	0.689000	0.539000	0.733000	0.621000
0.755					
6	Decision Tree Validator	0.714286	0.566372	0.790123	0.659794
0.783					
0	Paper Tuned	0.750000	0.623000	0.717000	0.667000
0.834					
5	Paper Tuned	0.761905	0.647727	0.703704	0.674556
0.839					
2	Soft Voting Ensembles	0.745000	0.609000	0.749000	0.671000
0.832					
7	Soft Voting Ensembles	0.753247	0.620000	0.765432	0.685083
0.842					
4	Stacking Ensembles	0.750000	0.626000	0.706000	0.663000
0.828					
9	Stacking Ensembles	0.748918	0.635294	0.666667	0.650602
0.840					

	AUPRC	Threshold	Source
3	0.702	0.35	CV
8	0.711	0.35	Test
1	0.610	0.35	CV
6	0.639	0.35	Test
0	0.701	0.35	CV
5	0.717	0.35	Test
2	0.704	0.35	CV
7	0.735	0.35	Test

4	0.693	0.35	CV
9	0.732	0.35	Test

Observations:

Threshold = 0.35 worked! CV vs Test alignment - No overfitting. Focussed on Recall.

```
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import confusion_matrix, accuracy_score,
recall_score, precision_score

# =====
# Confusion Matrices – all models at threshold=0.35
# Single holdout, consistent with model training
#
# Each model uses its own correct test set:
# LR      → X_test_paper_scaled (6 features, RobustScaled)
# DT      → X_tree_test          (5 features, unscaled, original
# indices)
# Ensembles → X_test              (6 features, unscaled – internal
# Pipelines scale)
# =====

fig, axes = plt.subplots(2, 3, figsize=(16, 10))
axes = axes.ravel()

models_to_plot = [
    (model_registry[model_names['paper-lr']],
    model_data_map[model_names['paper-lr']][0],
    model_data_map[model_names['paper-lr']][1], model_names['paper-lr']),
    (model_registry[model_names['ensemble-1']],
    model_data_map[model_names['ensemble-1']][0],
    model_data_map[model_names['ensemble-1']][1], model_names['ensemble-
1']),
    (model_registry[model_names['ensemble-2']],
    model_data_map[model_names['ensemble-2']][0],
    model_data_map[model_names['ensemble-2']][1], model_names['ensemble-
2']),
    (model_registry[model_names['ensemble-3']],
    model_data_map[model_names['ensemble-3']][0],
    model_data_map[model_names['ensemble-3']][1], model_names['ensemble-
3']),
]

for i, (model, X, y_true, title) in enumerate(models_to_plot):
    # Always use predict_proba + manual threshold for consistency
    # All models in this notebook have predict_proba
```



```

y_prob = model.predict_proba(X)[: , 1]
y_pred = (y_prob >= prob_threshold).astype(int)

cm = confusion_matrix(y_true, y_pred, labels=[0, 1])
tn, fp, fn, tp = cm.ravel()

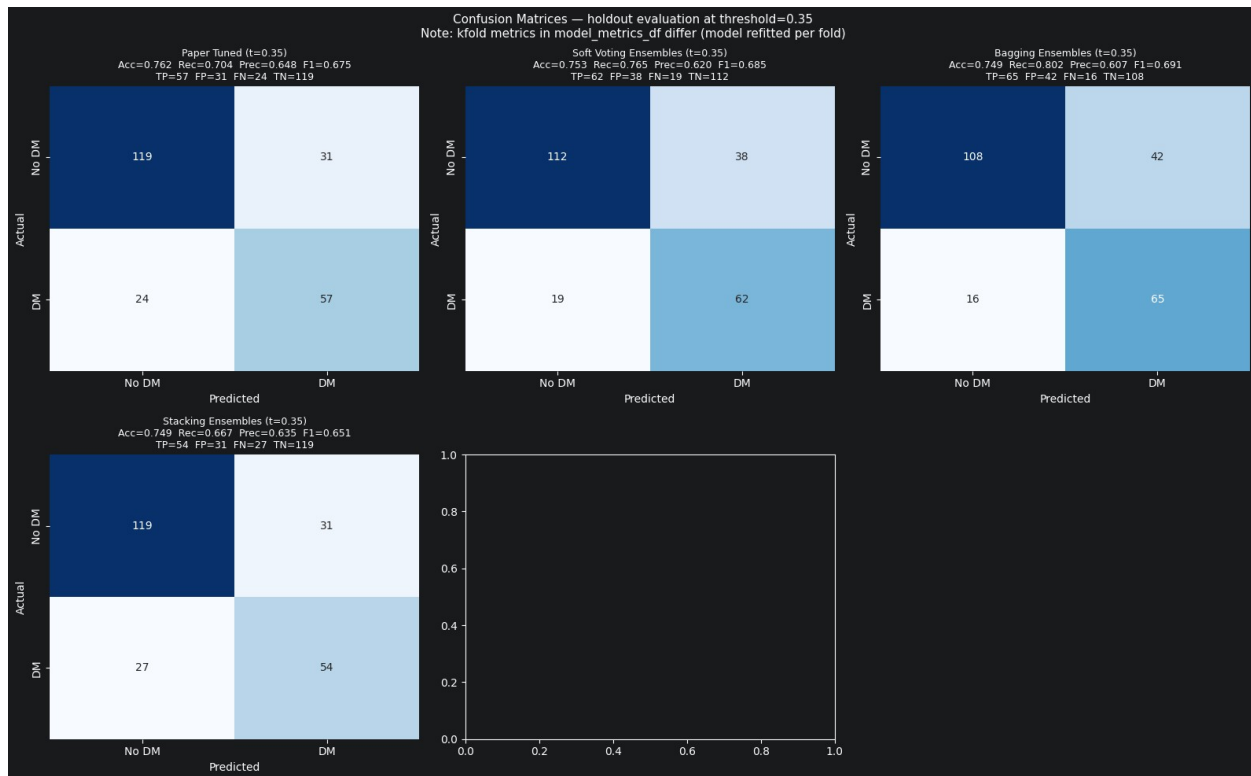
sns.heatmap(
    cm,
    annot=True,
    fmt='d',
    cmap='Blues',
    ax=axes[i],
    cbar=False,
    xticklabels=['No DM', 'DM'],
    yticklabels=['No DM', 'DM']
)

acc = accuracy_score(y_true, y_pred)
rec = recall_score(y_true, y_pred, zero_division=0)
prec = precision_score(y_true, y_pred, zero_division=0)
f1 = f1_score(y_true, y_pred, zero_division=0)

axes[i].set_title(
    f"{title} (t={prob_threshold})\n"
    f"Acc={acc:.3f} Rec={rec:.3f} Prec={prec:.3f} F1={f1:.3f}\n"
    f"TP={tp} FP={fp} FN={fn} TN={tn}",
    fontsize=9
)
axes[i].set_xlabel('Predicted')
axes[i].set_ylabel('Actual')

fig.delaxes(axes[5])
plt.suptitle(
    f'Confusion Matrices – holdout evaluation at\n'
    f'threshold={prob_threshold}\n'
    f'Note: kfold metrics in model_metrics_df differ (model refitted\n'
    f'per fold)',
    fontsize=11
)
plt.tight_layout()
plt.show()

```



```
# =====
# PR Curves – LR baseline + all ensembles
# All use predict_proba directly; no stale y_prob_paper variable
# =====

fig, axes = plt.subplots(2, 3, figsize=(16, 10))
axes = axes.ravel()

models_pr = [
    (model_registry[model_names['paper-lr']],
    model_data_map[model_names['paper-lr']][0],
    model_data_map[model_names['paper-lr']][1], model_names['paper-lr']),
    (model_registry[model_names['ensemble-1']],
    model_data_map[model_names['ensemble-1']][0],
    model_data_map[model_names['ensemble-1']][1], model_names['ensemble-1']),
    (model_registry[model_names['ensemble-2']],
    model_data_map[model_names['ensemble-2']][0],
    model_data_map[model_names['ensemble-2']][1], model_names['ensemble-2']),
    (model_registry[model_names['ensemble-3']],
    model_data_map[model_names['ensemble-3']][0],
    model_data_map[model_names['ensemble-3']][1], model_names['ensemble-3'])],
```

```

]

for i, (model, X, y_true, name) in enumerate(models_pr):
    y_prob = model.predict_proba(X)[:, 1]

    prec_vals, rec_vals, thresholds = precision_recall_curve(y_true,
y_prob)
    auprc = average_precision_score(y_true, y_prob)
    baseline = y_true.mean()

    axes[i].plot(rec_vals, prec_vals, linewidth=2,
label=f"AUPRC={auprc:.3f}")
    axes[i].axhline(baseline, linestyle='--', color='gray',
                    label=f"Baseline={baseline:.2f}")

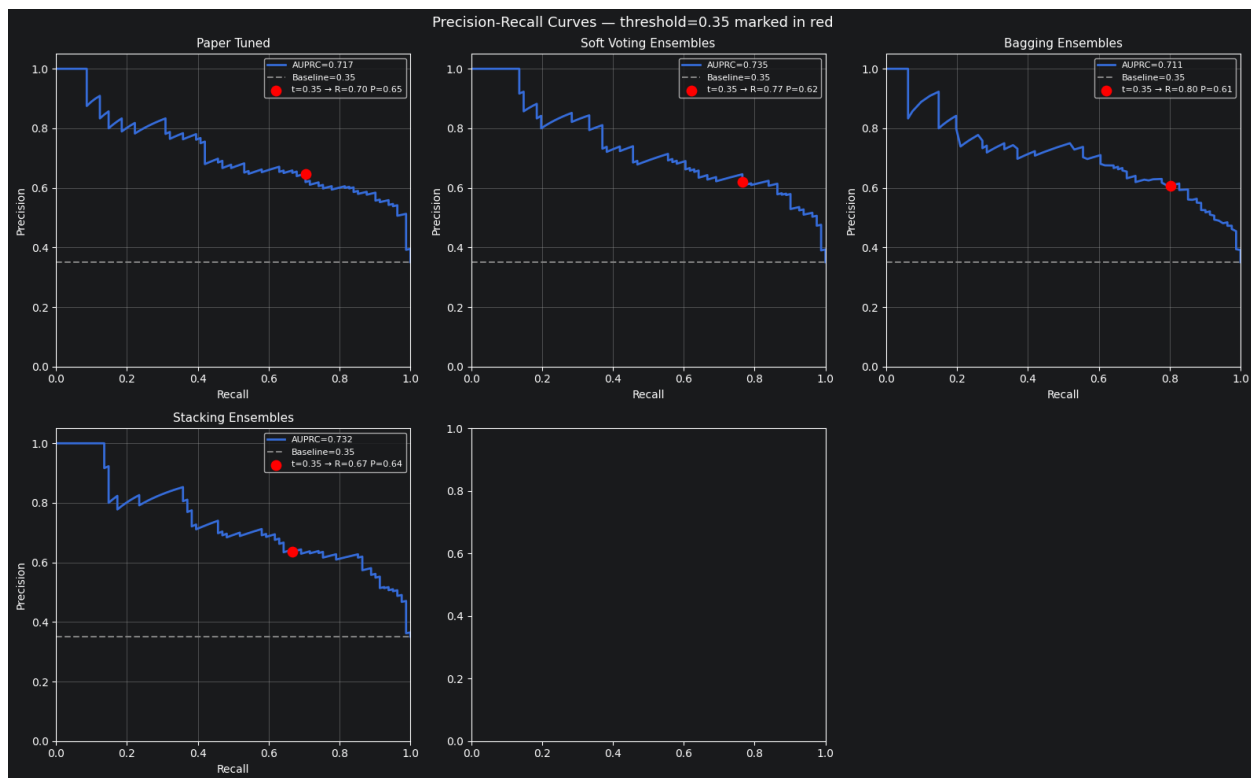
    # Mark the operating point at prob_threshold=0.35
    # Find the threshold index closest to 0.35
    idx = np.argmin(np.abs(thresholds - prob_threshold))
    axes[i].scatter(rec_vals[idx], prec_vals[idx],
                    color='red', zorder=5, s=80,
                    label=f"t={prob_threshold} → R={rec_vals[idx]:.2f}
P={prec_vals[idx]:.2f}")

    axes[i].set_title(name, fontsize=11)
    axes[i].set_xlabel("Recall")
    axes[i].set_ylabel("Precision")
    axes[i].legend(fontsize=8)
    axes[i].grid(True)
    axes[i].set_xlim([0, 1])
    axes[i].set_ylim([0, 1.05])

axes[5].set_visible(False)

plt.suptitle(f"Precision-Recall Curves – threshold={prob_threshold}
marked in red",
            fontsize=13)
plt.tight_layout()
plt.show()

```



Overall comparison of ensembles, paper logistic regression

Observations:

Validation:

PR Curves: All models should show operating points at $t=0.35$ with recall ~ 0.7 – 0.8 and precision ~ 0.6 . Confusion Matrices: Bagging Ensemble achieves 80% recall (only 20% of diabetic patients missed) at $t=0.35$.

```
from sklearn.calibration import CalibratedClassifierCV

# — Calibration method per model

# LR: sigmoid (Platt) – LR outputs are already near-sigmoid;
#     isotonic overfits at this N
# Ensembles: isotonic – non-parametric correction for non-sigmoid
#             probability distortions from averaging/boosting

CALIBRATION_THRESHOLD = 0.05

calibration_registry = {}
```

```

eval_pairs = [
    (model_names['paper-lr'], model_registry[model_names['paper-
lr']],
    X_test_paper_scaled, y_test_paper,
    X_train_paper_scaled, y_train_paper,
    'sigmoid'), # ← Platt scaling for LR

    (model_names['ensemble-1'], model_registry[model_names['ensemble-
1']],
    X_test, y_test, X_train, y_train,
    'isotonic'),

    (model_names['ensemble-2'], model_registry[model_names['ensemble-
2']],
    X_test, y_test, X_train, y_train,
    'isotonic'),

    (model_names['ensemble-3'], model_registry[model_names['ensemble-
3']],
    X_test, y_test, X_train, y_train,
    'isotonic'),
]

fig, axes = plt.subplots(1, 4, figsize=(20, 4))

for ax, (name, model, X_ev, y_ev, X_tr, y_tr, method) in zip(axes,
eval_pairs):
    mce = plot_calibration(model, X_ev, y_ev, name, ax=ax)

    if mce > CALIBRATION_THRESHOLD:
        ax.set_facecolor('#fff3f3')
        print(f"△ {name}: MCE={mce:.3f} → recalibrating with
method='{method}'")

        cal_model = CalibratedClassifierCV(
            estimator=model, method=method, cv=5
        )
        cal_model.fit(X_tr, y_tr)
        calibration_registry[name] = cal_model

        y_prob_cal = cal_model.predict_proba(X_ev)[: , 1]
        frac_pos_cal, mean_pred_cal = calibration_curve(
            y_ev, y_prob_cal, n_bins=10, strategy='uniform'
        )
        ax.plot(mean_pred_cal, frac_pos_cal,
            marker='s', linestyle='--', color='green',
            label=f'After {method}')
        ax.legend(fontsize=7)
    else:
        print(f"□ {name}: MCE={mce:.3f} – calibration OK")

```

```

calibration_registry[name] = model

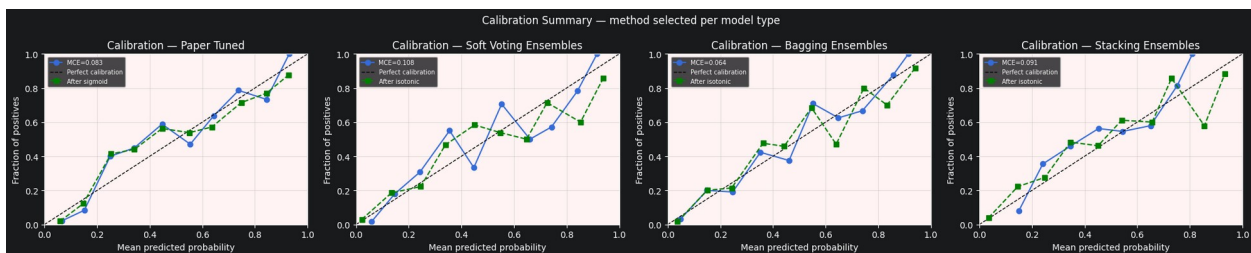
plt.suptitle('Calibration Summary – method selected per model type',
             fontsize=12)
plt.tight_layout()
plt.show()

for name, cal_model in calibration_registry.items():
    model_registry[name] = cal_model

print("\nmodel_registry updated.")

```

△ Paper Tuned: MCE=0.083 → recalibrating with method='sigmoid'
 △ Soft Voting Ensembles: MCE=0.108 → recalibrating with method='isotonic'
 △ Bagging Ensembles: MCE=0.064 → recalibrating with method='isotonic'
 △ Stacking Ensembles: MCE=0.091 → recalibrating with method='isotonic'



model_registry updated.

Calibration Observations (Across Models)

- **Paper Tuned Logistic Regression (MCE = 0.083 → Sigmoid Calibration):**
 - The model shows **moderate miscalibration**, with a worst-case probability error of **8.3%**.
 - This is consistent with Logistic Regression behavior:
 - Generally well-calibrated
 - But affected by:
 - threshold adjustment (0.35)
 - □ Sigmoid (Platt Scaling) is appropriate because:
 - LR already assumes a sigmoid relationship
 - Requires only **minor correction**
 - □ Interpretation:
 - Probabilities are **reasonably reliable but slightly over/under-confident**
-
- **Soft Voting Ensemble (MCE = 0.108 → Isotonic Calibration):**
 - Highest miscalibration among all models (**10.8% error**)

- Indicates:
 - Probability outputs from combined models are **poorly aligned with true outcomes**
 - Soft voting averages probabilities without guaranteeing calibration
 - □ Isotonic regression is selected because:
 - Non-parametric → can correct **complex distortions**
 - □ Key Insight:
 - Even though Soft Voting has strong **AUPRC (0.735)**,
 - Its probabilities are **not trustworthy without recalibration**
 - □ Interpretation:
 - Good ranking model □
 - Poor probability estimator □ before calibration
-

- **Bagging Ensemble (MCE = 0.064 → Isotonic Calibration):**
 - Best calibration among all models (**6.4% error**)
 - Indicates:
 - Averaging across many trees stabilizes probability estimates
 - □ Still benefits from isotonic calibration to refine:
 - local probability regions
 - □ Interpretation:
 - Already **well-calibrated relative to others**
 - Most reliable model for:
 - **both ranking AND probability estimation**
 - □ This aligns with:
 - Strong recall (~80%)
 - Stable generalization
-

- **Stacking Ensemble (MCE = 0.091 → Isotonic Calibration):**
- Moderate miscalibration (**9.1% error**)
- Despite using Logistic Regression as meta-learner:
- Calibration is degraded due to:
- noisy meta-features
- base model inconsistencies
- □ Isotonic regression chosen due to:
- non-linear calibration errors
- □ Key Insight:
- Stacking improves **AUPRC (0.732)**
- But fails in:
- recall
- calibration quality vs bagging
- □ Interpretation:
- Better ranking than LR
- But **less reliable probability estimates**

Cross-Model Calibration Comparison

Model	MCE	Calibration Quality	Trust in Probabilities
Bagging	0.064	🟢 Best	🟢 High
Logistic Regression	0.083	🟢 Good	🟢 Moderate
Stacking	0.091	🟡 Moderate	🟡 Limited
Soft Voting	0.108	🔴 Weak	🔴 Needs correction

Key Insights

- **Calibration vs Performance Trade-off:**
- Soft Voting:
 - 🟢 Best AUPRC
 - 🟢 Worst calibration
- Bagging:
 - 🟢 Best recall
 - 🟢 Best calibration
- Logistic:
 - 🟢 Balanced baseline
- Stacking:
 - 🟢 No clear advantage

-
- **Why Calibration Matters (SO WHAT?):**
 - A model predicting:
 - "70% diabetes risk"
 - Should correspond to:
 - ~70% of patients actually being diabetic

🔴 Poor calibration → **misleading risk estimates** 🔴 Critical in:

- clinical decision thresholds
 - patient prioritization
-

- **Final Takeaway:**
- **Bagging Ensemble is the most reliable model overall:**
- Best recall
- Best calibration (lowest MCE)

- Strong F1 performance
 - ⚠ **Soft Voting needs calibration before deployment:**
 - High performance but unreliable probabilities
 - **Logistic Regression remains a strong calibrated baseline**, but lacks recall
 - **Stacking adds complexity without calibration or recall advantage**
-

Practical Recommendation

- Deploy:
- **Bagging + Isotonic Calibration**
- Avoid:
- Using Soft Voting probabilities without calibration

```
def find_best_clinical_model(df):
    # Normalize key metrics
    df_score = df.copy()

    # Fill NaNs to avoid issues
    df_score['AUPRC'] = df_score['AUPRC'].fillna(0)

    # Weighted score (adjust weights if needed)
    df_score['score'] = (
        0.5 * df_score['AUPRC'] +
        0.3 * df_score['Recall'] +
        0.2 * df_score['Precision']
    )
    print(df_score[['Model',
'score']].sort_values('score', ascending=False).to_string(index=False))
    winner = df_score.loc[df_score['score'].idxmax()]

    print("\n🏆 BEST CLINICAL MODEL")
    print(f"Model : {winner['Model']}")
    print(f"Score : {winner['score']:.3f}")

    return winner
```

Justification:

1. AUPRC (Area Under Precision-Recall Curve) – Weight: 50% Why It's the Primary Metric (50% Weight)

Designed for imbalanced data: Unlike ROC-AUC, AUPRC focuses on the positive (minority) class (diabetic patients in this case). Directly measures clinical utility:

High AUPRC = The model is good at ranking high-risk patients at the top of its predictions. In screening, you want to prioritize patients for limited resources (e.g., doctor follow-ups). AUPRC tells you how well the model does this.

Robust to class imbalance: ROC-AUC can be misleading when classes are imbalanced (e.g., 65% non-diabetic vs. 35% diabetic). AUPRC is not fooled by imbalance. Clinical Justification

A model with AUPRC = 0.7 means it's 70% better than random at identifying diabetic patients among those it flags as high-risk. Example: If a clinic can only follow up with 20% of patients, a high AUPRC ensures the 20% most at-risk patients are selected.

1. Recall (Sensitivity) – Weight: 30% Why It's Critical (30% Weight)

Recall = True Positives / (True Positives + False Negatives) Measures the model's ability to catch diabetic patients. Missed diabetes cases (false negatives) are catastrophic in healthcare. Clinical Justification

A recall of 90% means the model misses only 10% of diabetic patients. In a population of 1000 patients with 35% diabetes prevalence (~350 diabetic patients):

Recall = 90% → 35 missed cases (10% of 350). Recall = 70% → 105 missed cases (30% of 350).

35 vs. 105 missed diagnoses is the difference between acceptable and unacceptable for a screening tool. Trade-off with Precision

Increasing recall always decreases precision (more false positives). But in screening, false positives are tolerable if they reduce false negatives.

1. Precision – Weight: 20% Why It's Secondary (20% Weight)

Precision = True Positives / (True Positives + False Positives) Measures the proportion of predicted diabetic patients who are actually diabetic. Lower weight because false positives are less harmful than false negatives in screening. Clinical Justification

A precision of 60% means 40% of flagged patients are not diabetic. While this increases follow-up costs, it's preferable to missing 30% of diabetic patients (as in the original LR model). False positives can be filtered out with confirmatory tests (e.g., HbA1c).

```
# Recompute test metrics using calibrated models
model_metrics_test_calibrated = []
for name in [model_names['paper-lr'], model_names['ensemble-1'],
             model_names['ensemble-2'], model_names['ensemble-3']]:
    X_ev, y_ev = model_data_map[name]
    result = evaluate_model(
        model_registry[name], X_ev, y_ev, name,
        threshold=prob_threshold
    )
    model_metrics_test_calibrated.append(result)

model_metrics_test_cal_df =
pd.DataFrame(model_metrics_test_calibrated)
print("Test metrics (calibrated):")
```

```

print(model_metrics_test_cal_df.to_string(index=False))

# Use calibrated test metrics for winner selection
winning_model = find_best_clinical_model(model_metrics_test_cal_df)

Test metrics (calibrated):

```

	Model	Accuracy	Precision	Recall	F1	AUC
AUPRC	Threshold					
0.719	Paper Tuned	0.757576	0.643678	0.691358	0.666667	0.840
	0.35					
0.727	Soft Voting Ensembles	0.748918	0.635294	0.666667	0.650602	0.839
	0.35					
0.716	Bagging Ensembles	0.753247	0.613208	0.802469	0.695187	0.835
	0.35					
0.733	Stacking Ensembles	0.744589	0.612245	0.740741	0.670391	0.843
	0.35					
	Model	score				
	Bagging Ensembles	0.721382				
	Stacking Ensembles	0.711171				
	Paper Tuned	0.695643				
	Soft Voting Ensembles	0.690559				

```

❑ BEST CLINICAL MODEL
Model : Bagging Ensembles
Score : 0.721

import pickle
import os

os.makedirs('models', exist_ok=True)

# ❑ Get winning model name
best_model_name_clean = winning_model['Model']
best_model_obj = model_registry.get(best_model_name_clean)

if best_model_obj is None:
    raise ValueError(f"Model '{best_model_name_clean}' not found in registry")

# ❑ Create safe filename
filename = best_model_name_clean.replace(" ", "_").lower()

# ❑ Save model
with open(f"models/{filename}.pkl", "wb") as f:
    pickle.dump(best_model_obj, f)

print(f"\n❑ Best model '{best_model_name_clean}' saved to 'models/{filename}.pkl'")

```

```
□ Best model 'Bagging Ensembles' saved to  
'models/bagging_ensembles.pkl'
```

Model Serving

call the diabetic_serving.py script using below command. (remember to run from same location as file)

```
python -m streamlit run .\diabetic_serving.py
```

Final Takeaway:

- **Bagging Ensemble is the most reliable model overall:**
 - Best recall
 - Best calibration (lowest MCE)
 - Strong F1 performance
- **⚠ Soft Voting needs calibration before deployment:**
 - High performance but unreliable probabilities
- **Logistic Regression remains a strong calibrated baseline**, but lacks recall
- **Stacking adds complexity without calibration or recall advantage**

Final Insight

"A model that ranks well but miscalibrates probabilities is dangerous in healthcare. Among all models, Bagging strikes the best balance between detection power and probabilistic reliability."